

RWD

Real-Time Weapon Detection

A Senior Project Report

Prepared by

Abdullah Abdalrhman Alzhnan

Supervised by

Dr. Nisreen Sulayman

Eng. Zeinab Baghdadi

2025-2026

Abstract

The proliferation of surveillance camera networks has dramatically outpaced the human capacity to monitor them in real time. Critical security events involving weapons such as firearms and bladed objects are frequently missed, delayed, or recognised only after harm has been inflicted. This project, named Real-Time Weapon Detection (RWD), proposes an automated visual surveillance system that uses a deep learning-based object detection framework to identify weapons in live video streams and immediately raise an audible alarm to alert security personnel.

The system is built upon the YOLOv8 single-stage object detector, which is fine-tuned on a custom dataset assembled from publicly available Roboflow Universe collections. Seven weapon categories are addressed in the present version of the system: Celurit, Golok, Kapak, Pedang, Pisau, Pistol and Senapan. The dataset was preprocessed to a uniform 640×640 resolution, automatically annotated in YOLO format and divided into training, validation and testing subsets. Two complete training experiments were carried out. The first served as a baseline using moderate augmentation and a fixed learning rate, whereas the second introduced more aggressive geometric and photometric augmentation, MixUp regularisation, a cosine learning-rate schedule and an early-stopping criterion in order to obtain a more general model.

Quantitative evaluation on the validation set demonstrates that the fine-tuned model achieves a mean Average Precision at intersection-over-union threshold 0.5 of 0.943, with overall precision of 0.939 and recall of 0.881. These figures represent a measurable improvement over the baseline, which obtained a mean Average Precision of 0.926 under identical conditions. To translate these offline results into a deployable surveillance tool, a real-time inference pipeline was implemented in Python using OpenCV and the Ultralytics library. The pipeline incorporates a temporal smoothing rule that requires two consecutive positive detections before the alarm is raised, which markedly reduces the rate of false positives that would otherwise occur in cluttered scenes.

The contributions of this project are threefold. First, it demonstrates that modern single-stage detectors can reach competitive accuracy on a heterogeneous, multi-class weapon dataset using only consumer-grade hardware. Second, it shows that simple temporal heuristics, when combined with a strong frame-level detector, can yield a robust real-time alerting system. Third, it provides a reproducible end-to-end methodology covering dataset assembly, model training, evaluation and deployment, and identifies clear directions for future work, including richer temporal modelling, low-light robustness and integration with operational closed-circuit television networks.

الملخص

شهد العالم في السنوات الأخيرة انتشاراً واسعاً لأنظمة المراقبة بالكاميرات في الأماكن العامة والخاصة، إلا أن الاعتماد على المراقبة البشرية المباشرة لهذه الكاميرات يبقى محدوداً وغير فعال أمام الكم الهائل من المقاطع المصورة التي تُنتج يومياً. وتشير الدراسات إلى أن المراقبين البشريين قد يفوتون نسبة كبيرة من الأحداث الحرجة خلال الجلسات الطويلة، وهو ما يفتح المجال أمام الذكاء الاصطناعي ليقدم حلولاً تلقائية تساعد في رصد التهديدات في الوقت الفعلي.

يقترح هذا المشروع، الذي يحمل اسم RWD اختصاراً لـ Real-Time Weapon Detection، نظاماً ذكياً للكشف الآني عن الأسلحة في تدفقات الفيديو، بحيث يُصدر النظام إنذاراً صوتياً فوراً عند اكتشاف سلاح في المشهد. يعتمد النظام على نموذج YOLOv8 للكشف عن الأجسام، وقد جرى تدريبه وضبطه الدقيق على مجموعة بيانات مخصصة تحتوي على سبعة أصناف من الأسلحة هي: Senapan و Pistol و Pisau و Pedang و Kapak و Golok و Celurit.

أُجريت تجربتان تدريبيتان متكاملتان: الأولى تجربة أساسية بمعاملات اعتيادية، والثانية تجربة ضبط دقيق طُبِّقت فيها تقنيات تعزيز بيانات أكثر تطوراً مثل MixUp و Mosaic، إلى جانب جدول تعلّم جيب التمام والإيقاف المبكر. وقد حقّق النموذج النهائي دقة متوسطة قدرها 0.943 عند عتبة 0.5 لمعيار IoU، ودقة عامة قدرها 0.939، واستدعاء قدره 0.881، وهي نتائج تتجاوز التجربة الأساسية بفارق ملحوظ.

جرى دمج النموذج المُدرَّب ضمن خط معالجة فعلي مكتوب بلغة Python ومعتمد على مكتبة OpenCV، حيث تُقرأ صور الكاميرا إطاراً تلو الآخر، ويمرّر كل إطار ثالث على النموذج لتحقيق أداء فوري، مع اشتراط ظهور الكشف في إطارين متتاليين قبل تشغيل الإنذار، وذلك للحدّ من الإنذارات الكاذبة. ويُظهر النظام بمجملة جدوى استخدام تقنيات الرؤية الحاسوبية الحديثة كأداة داعمة للأنظمة الأمنية، ويقدم منهجية متكاملة قابلة للتطوير لاحقاً نحو نمذجة زمنية أعمق ودمج مع شبكات المراقبة التشغيلية.

Acknowledgements

First and foremost, I would like to express my sincere gratitude to my supervisors, Dr. Nisreen Sulayman and Eng. Zeinab Baghdadi, for their continuous guidance, valuable feedback and unwavering support throughout the development of this project. Their academic insight, constructive criticism and encouragement have been essential in shaping the direction and quality of this work.

I would also like to thank the Faculty of Artificial Intelligence Engineering at the Syrian Private University, and in particular the Department of Artificial Intelligence and Data Science, for providing a stimulating academic environment and for equipping me with the theoretical and practical foundations on which this project is built.

Special thanks are due to my colleagues and classmates, whose discussions and collaboration helped clarify many of the technical challenges encountered during dataset preparation, model training and system integration. Their willingness to share ideas and review intermediate results has been a constant source of motivation.

Finally, I am profoundly grateful to my family, whose patience, encouragement and confidence in my abilities made the long hours of research, training and writing possible. This work is dedicated to them.

Table of Contents

Chapter 1. Introduction

1.1 General Introduction.....	11
1.2 Motivation.....	11
1.3 Problem Definition.....	12
1.4 Objectives of the Project.....	12
1.5 Project Scope and Limitations.....	13
1.6 Importance of the Study.....	14
1.7 Report Organisation.....	14
1.8 Tools and Technologies.....	14

Chapter 2. Background and Related Work

2.1 Introduction.....	17
2.2 The Need for Automated Surveillance.....	17
2.3 Computer Vision and Deep Learning.....	17
2.4 Object Detection Paradigms.....	18
2.5 The YOLO Family of Detectors.....	19
2.6 Related Work in Weapon Detection.....	19
2.7 Case Studies.....	20
2.8 Research Gap.....	22

Chapter 3. Theoretical Foundations

3.1 Introduction.....	24
3.2 Convolutional Neural Networks.....	24
3.3 The Object Detection Problem.....	25
3.4 The YOLOv8 Architecture.....	25
3.5 Loss Functions in YOLOv8.....	26
3.6 Optimisation and Learning-Rate Scheduling.....	27
3.7 Data Augmentation Theory.....	28
3.8 Non-Maximum Suppression.....	28

Chapter 4. System Overview and Methodology

4.1 Introduction.....	31
4.2 System Architecture.....	31
4.3 Choice of Detection Model.....	31
4.4 Frame-Based Video Processing.....	32
4.5 Temporal Smoothing for Alarm Stability.....	33

4.6 Real-Time Processing Considerations.....	33
4.7 System Workflow.....	34
Chapter 5. Dataset Preparation	
5.1 Introduction.....	37
5.2 Dataset Source and Description.....	37
5.3 Class Definitions.....	37
5.4 Dataset Distribution.....	38
5.5 Annotation Strategy.....	39
5.6 Preprocessing Pipeline.....	40
5.7 Data Augmentation.....	40
5.8 Dataset Splitting Strategy.....	42
Chapter 6. Model Training and Experiments	
6.1 Introduction.....	45
6.2 Training Environment.....	45
6.3 Experiment 1: Baseline Training.....	45
6.4 Experiment 2: Fine-Tuning.....	46
6.5 Evaluation Metrics.....	48
6.6 Comparative Summary.....	50
Chapter 7. Results and Discussion	
7.1 Introduction.....	52
7.2 Quantitative Results.....	52
7.3 Per-Class Analysis.....	52
7.4 Confusion Matrix Analysis.....	53
7.5 Precision-Recall and F1 Curves.....	55
7.6 Qualitative Results.....	57
7.7 Real-Time Deployment.....	59
7.8 Limitations.....	59
Chapter 8. Conclusion and Future Work	
8.1 Conclusion.....	62
8.2 Future Work.....	63
Chapter 9. References	
9.1 References.....	66

Table of Figures

Figure 4.1: System pipeline of the RWD weapon detection system.....	35
Figure 5.1: Training batch sample from the dataset.....	41
Figure 5.2: Augmented training batch from the fine-tuning experiment.....	42
Figure 6.1: Baseline experiment training and validation curves.....	46
Figure 6.2: Fine-tuning experiment training and validation curves.....	47
Figure 6.3: Performance metrics across baseline epochs.....	49
Figure 6.4: Loss curves of the baseline experiment.....	49
Figure 7.1: Normalised confusion matrix for the baseline experiment.....	53
Figure 7.2: Normalised confusion matrix for the fine-tuned model.....	54
Figure 7.3: Precision-Confidence curve for all classes.....	55
Figure 7.4: Precision-Recall curve for all classes.....	55
Figure 7.5: F1-Confidence curve for all classes.....	56
Figure 7.6: Qualitative validation predictions of the fine-tuned model.....	57

Table of Shortcuts

RWD	Real-Time Weapon Detection
AI	Artificial Intelligence
CNN	Convolutional Neural Network
YOLO	You Only Look Once
mAP	Mean Average Precision
IoU	Intersection over Union
FPS	Frames Per Second
R-CNN	Region-based Convolutional Neural Network
SSD	Single Shot MultiBox Detector
DFL	Distribution Focal Loss
CIoU	Complete Intersection over Union
SGD	Stochastic Gradient Descent
AdamW	Adam with decoupled Weight decay
CCTV	Closed-Circuit Television
ReLU	Rectified Linear Unit
NMS	Non-Maximum Suppression
GPU	Graphics Processing Unit
ASD	(Reference convention only)

Table of Tables

Table 2.1. Case Study 1 — YOLOv5-based Violence Detection.....	20
Table 2.2. Case Study 2 — YOLO Anomaly Activity Detection.....	21
Table 2.3. Case Study 3 — CNN-YOLO Face and Weapon Detection.....	21
Table 5.1. Class definitions used in the RWD dataset.....	37
Table 5.2. Approximate dataset distribution per class.....	38
Table 5.3. Augmentation pipeline for Experiment 1 (Baseline).....	40
Table 5.4. Augmentation pipeline for Experiment 2 (Fine-Tuning).....	41
Table 6.1. Hyperparameters of the baseline experiment.....	45
Table 6.2. Hyperparameters of the fine-tuning experiment.....	47
Table 6.3. Comparison of overall metrics between experiments.....	50
Table 7.1. Per-class Average Precision of the fine-tuned model.....	53

Chapter 1: Introduction

1.1 General Introduction

Public and private spaces around the world are increasingly equipped with networks of surveillance cameras. Train stations, airports, schools, shopping centres, factories and government buildings now routinely operate fleets of cameras that produce continuous video streams. Yet the very ubiquity of these cameras has made human-only monitoring impractical. A single operator can be expected to watch only a small number of feeds attentively, and even those for a limited period before fatigue and inattention degrade performance. As a consequence, a large fraction of what is recorded is never examined in real time, and is consulted only retrospectively, after an incident has already occurred.

Among the events that surveillance systems are most often called upon to detect, the appearance of a weapon is one of the most consequential. The presence of a firearm or a sharp bladed object in a public space is, in many cases, an immediate precursor to violence, and the time gap between visibility and harm is typically very short. An automated system capable of recognising weapons the moment they enter a camera's field of view, and of issuing an alert before any harmful action takes place, would therefore add a layer of protection that purely human-monitored systems cannot provide.

The Real-Time Weapon Detection (RWD) project addresses precisely this need. The system processes a live video stream, applies a deep learning-based object detector to each frame and raises an audible alarm whenever a weapon is detected with sufficient confidence. The detector itself is a fine-tuned variant of the YOLOv8 family of single-stage convolutional neural networks, chosen for its favourable balance between accuracy and inference speed. The training data covers seven categories of weapons of varying shape, size and visual complexity, and the trained model is integrated with a lightweight Python pipeline that operates on standard consumer hardware.

This report documents the full lifecycle of the RWD project, from problem identification and dataset construction through to model training, evaluation and real-time deployment. It is intended both as a record of the technical decisions taken during the project and as a guide for future students or practitioners who wish to extend the work.

1.2 Motivation

The motivation for this project is grounded in three distinct but converging observations. The first is the documented inadequacy of purely human surveillance for detecting rare but high-stakes events. Studies on operator vigilance consistently report that the rate at which target events are missed grows steeply with the number of feeds being watched and with the duration of the monitoring shift. The second is the accelerating maturity of deep learning-based object detection, which has crossed the threshold from research curiosity to deployable technology in domains as diverse as autonomous driving, medical imaging and industrial quality control. The third is the comparatively small body of published work on the specific problem of real-time, multi-class weapon detection in unconstrained surveillance footage.

Most of the existing literature on weapon detection focuses either on firearms alone, on still images rather than video streams, or on tightly constrained datasets that fail to reflect the diversity of real-world environments. There is also relatively little discussion of the engineering decisions that arise when a frame-level detector is embedded in a live alerting system. Issues such as the choice of input resolution, the spacing of inference calls, the suppression of spurious single-frame detections and the strategy used to drive an alarm without overwhelming the operator are often left implicit.

The present project is motivated by the desire to address this gap in a concrete and reproducible manner. By choosing a relatively wide set of weapon classes, by training on heterogeneous data drawn from public sources, and by reporting both the offline accuracy of the model and the behaviour of the deployed pipeline, the RWD project aims to produce a piece of work that is useful in practice and informative for further research.

1.3 Problem Definition

The core problem that this project sets out to solve can be stated as follows. Given a continuous stream of video frames captured by a camera observing an indoor or outdoor scene, the system must determine, for each frame and in close to real time, whether one or more weapons of interest are visible. When a weapon is detected with sufficient confidence and persistence, the system must issue an audible alarm intended to draw the attention of a human operator or, in deployments without an operator, to deter the bearer.

Several aspects of this problem deserve emphasis. The detection task is multi-class rather than binary: the system must distinguish among seven weapon categories and must also reject every other object that may appear in the scene, ranging from harmless tools to background clutter. The detection must occur in real time, which constrains both the size of the model and the rate at which frames can be processed. The system must be robust against variations in illumination, viewpoint, scale and partial occlusion, since none of these conditions can be controlled at deployment time. Finally, the system must avoid raising false alarms whenever possible, because frequent false alarms quickly erode operator trust and ultimately render the system useless.

These requirements together define a challenging engineering problem that combines elements of computer vision, deep learning and real-time systems design. The remainder of this report describes how each of these elements was addressed.

1.4 Objectives of the Project

The objectives of the project, derived from the problem definition above, are organised into a primary objective and a set of supporting objectives. The primary objective is to design, implement and evaluate an end-to-end system that performs accurate, real-time detection of weapons in video streams and that responds to such detections by raising an audible alarm.

The supporting objectives are summarised in the following list:

- To assemble a multi-class weapon detection dataset of sufficient size and diversity to support meaningful supervised training, drawing on publicly available sources and respecting their licensing terms.
- To prepare and preprocess the dataset, including resizing, normalisation, annotation verification and the application of carefully chosen data augmentation strategies.
- To train and fine-tune a YOLOv8-based object detector on this dataset, conducting at least two complete experiments with different training configurations to allow direct comparison.
- To evaluate the trained model using the standard quantitative metrics for object detection, namely precision, recall and mean Average Precision, as well as per-class metrics and confusion matrices.
- To integrate the best-performing model into a real-time inference pipeline implemented in Python, using the OpenCV library for video capture and rendering.
- To incorporate a temporal smoothing mechanism that reduces the rate of false alarms by requiring repeated detection across successive frames before triggering the alert.
- To document the methodology, results and limitations of the project in a manner that supports both reproducibility and future extension.

1.5 Project Scope and Limitations

The scope of the project is intentionally bounded in several respects. First, the system is confined to visual data captured in the visible spectrum by ordinary RGB cameras. It does not exploit thermal, depth or radar sensors, although such modalities are known to be useful in certain surveillance scenarios. Second, the detector operates on individual frames and uses only short-range temporal information through a simple consecutive-detection rule. Sophisticated temporal architectures such as 3D convolutions, recurrent networks or transformer-based video models are out of scope for this iteration.

Third, the dataset on which the system is trained, although diverse, is finite. It cannot be expected to capture every weapon, every viewpoint or every environmental condition under which the system might eventually be deployed. The reported accuracy figures, therefore, refer to the validation distribution and should be interpreted as indicative rather than as guarantees of field performance. Fourth, the deployment target is a single workstation equipped with a webcam or a similar capture device, and the system has not been tested at the scale of large multi-camera installations, although its architecture would support such an extension.

Finally, it is important to clarify that the system is intended as a decision-support tool rather than as an autonomous security mechanism. Its outputs are alerts, not actions; the responsibility for verifying the alert and deciding on a response rests entirely with the human operator. This

positioning, far from being a weakness, is in line with current best practice for the deployment of artificial intelligence in safety-critical domains.

1.6 Importance of the Study

The importance of the present study can be appreciated from several perspectives. From a societal perspective, any reliable improvement in the speed and consistency with which weapons are detected in surveillance footage has the potential to reduce harm and save lives. Even a marginal reduction in the time between weapon visibility and operator response can make the difference between a contained incident and an escalation. From an academic perspective, the project contributes a complete, documented case study of the application of modern object detection methods to a multi-class weapon recognition task, including the often-underreported step of moving from offline benchmarks to a deployed real-time pipeline.

From an engineering perspective, the project illustrates how off-the-shelf components such as Ultralytics YOLOv8, OpenCV and Roboflow can be combined into a coherent system without the need for bespoke deep-learning frameworks. This lowers the barrier to entry for similar projects and allows future students to focus on the harder problems of dataset quality, evaluation and deployment rather than on infrastructure. Finally, from a pedagogical perspective, the project consolidates a wide range of topics covered in the curriculum of the Department of Artificial Intelligence and Data Science, including convolutional neural networks, optimisation, evaluation methodology, software engineering and the ethical dimensions of automated decision making.

1.7 Report Organisation

The remainder of this report is organised into eight further chapters. Chapter 2 reviews the technical and academic background of the project, surveying related work and identifying the gap that this project aims to fill. Chapter 3 presents the theoretical foundations on which the system is built, including convolutional neural networks, the object detection problem and the architecture of YOLOv8. Chapter 4 describes the overall system design and methodology, while Chapter 5 documents the construction and preparation of the dataset.

Chapter 6 reports on the two training experiments carried out during the project, presenting the configurations, the training dynamics and the resulting models. Chapter 7 analyses the experimental results in detail, both quantitatively and qualitatively, and discusses the limitations identified during evaluation. Chapter 8 concludes the report by summarising the contributions of the project and outlining promising directions for future work. Chapter 9 collects all bibliographic references.

1.8 Tools and Technologies

The implementation of the RWD system relies on a stack of open-source tools and libraries, each chosen for its maturity, community support and suitability for the problem at hand. Python served

as the principal programming language, given its dominant position in the deep learning community and the breadth of its scientific computing ecosystem. The Ultralytics implementation of YOLOv8 was used as the object detection framework, as it provides a high-level API for training, validation and inference, while exposing the configuration knobs needed for serious experimentation.

PyTorch underpins the Ultralytics framework and provides the automatic differentiation and GPU acceleration on which all training depends. OpenCV was used for capturing frames from the camera, decoding video files, drawing bounding boxes and overlaying textual information on the displayed feed. NumPy and pandas supported all numerical and tabular data manipulation, while Matplotlib was used to generate the various plots that appear in the results chapter. Roboflow served as the dataset management platform, providing tools for dataset assembly, annotation review, augmentation and export in YOLO-compatible format.

Training was carried out on Google Colab, which offered access to a graphics processing unit at no cost and integrated cleanly with the project's storage on Google Drive. Version control was maintained through Git, and the source code of the inference pipeline was kept in a single Python script for ease of deployment. The combination of these tools formed a workflow that was simultaneously powerful enough to support the technical demands of the project and lightweight enough to be reproducible by any future student with a comparable computing environment.

Chapter 2: Background and Related Work

2.1 Introduction

This chapter provides the contextual background necessary to motivate and understand the technical choices made in the remainder of the report. It begins with a brief account of why automated surveillance has become a topic of urgent practical interest, then introduces the relevant developments in computer vision and deep learning that make the present project feasible. The chapter then narrows its focus to the family of object detection algorithms to which YOLOv8 belongs, surveys representative work in the specific area of weapon detection, presents three case studies in detail and finally identifies the research gap that this project aims to address.

2.2 The Need for Automated Surveillance

Surveillance, as a social and technical practice, has expanded enormously over the past three decades. The cost of cameras and digital storage has fallen by several orders of magnitude, while the spread of broadband networks has made it routine to transmit dozens or hundreds of video streams from remote locations to a central monitoring station. The number of cameras installed in major cities is now estimated in the millions, and the volume of video produced daily by these networks far exceeds the capacity of human operators to watch in real time.

The mismatch between data volume and human attention has well-documented consequences. Research in the human factors literature shows that the probability of detecting a rare event in a video feed declines steeply with both the number of simultaneous feeds being monitored and the elapsed time on shift. Operators may miss a substantial proportion of critical events even under controlled conditions, and the rate of missed events grows further when feeds are noisy, when the events of interest are visually subtle, or when the operator's workload involves tasks beyond mere observation. These limitations of human vigilance have long been understood by security professionals, and they constitute the principal practical justification for automated assistance.

Within the broad space of automated surveillance, weapon detection occupies a particularly demanding niche. Unlike applications such as licence-plate recognition or simple intrusion detection, weapon detection involves objects that are highly variable in shape, that appear at a wide range of scales and viewpoints, and that may be partially occluded by the body of the bearer or by other objects in the scene. The cost of a missed detection is potentially severe, while the cost of a false detection, although less catastrophic, is sufficient to undermine operator trust if it occurs frequently. These competing requirements drive the technical design choices documented in later chapters.

2.3 Computer Vision and Deep Learning

Computer vision is the branch of artificial intelligence concerned with the extraction of meaningful information from digital images and video. For most of its history, the field relied on hand-engineered features such as edges, corners, histograms of oriented gradients and scale-invariant keypoints, combined with classical machine learning classifiers. While capable of impressive

results in narrow domains, hand-engineered approaches typically failed to scale to the variability of real-world data and required substantial expert effort for each new task.

The advent of deep learning, and in particular of convolutional neural networks trained on large image collections, transformed the field. Beginning around 2012, deep convolutional models began to outperform hand-engineered pipelines on standard benchmarks by margins that grew with each successive generation of architectures. The key insight was that the features required for visual recognition could be learned directly from data, given sufficient training examples, sufficient computational resources and an appropriate inductive bias in the form of convolutional weight sharing and hierarchical pooling. Today, deep convolutional networks form the backbone of essentially every state-of-the-art image classification, detection and segmentation system.

For object detection specifically, deep learning has driven a sustained reduction in error rates and a steady improvement in inference speed. Early deep detectors operated at a fraction of a frame per second, while modern single-stage detectors comfortably exceed thirty frames per second on commodity hardware while delivering accuracy that would have been considered state of the art only a few years earlier. This combination of speed and accuracy is what makes the present project technically feasible and is the principal reason for adopting a YOLO-family detector as the core of the RWD system.

2.4 Object Detection Paradigms

Object detection differs from image classification in that the system must not only assign a category label to an image but also indicate the location and extent of every instance of every category of interest. The output of a detector is therefore a set of bounding boxes, each with an associated category label and confidence score, rather than a single label per image.

Modern deep object detectors fall broadly into two families. Two-stage detectors, of which the R-CNN lineage is the most prominent example, first generate a set of candidate regions in the image and then classify and refine each candidate independently. Two-stage detectors typically achieve excellent accuracy but at a substantial computational cost, since each candidate region requires a separate forward pass through the classifier head. They are well suited to offline analysis but are often too slow for real-time deployment on consumer hardware.

Single-stage detectors, by contrast, predict bounding boxes and class labels in a single pass over the image. The YOLO family, the SSD family and more recent designs such as RetinaNet and FCOS all belong to this category. Single-stage detectors achieve their speed advantage by treating detection as a dense regression problem over a fixed grid of spatial locations, and by sharing the bulk of their computation across all candidate detections. The trade-off is typically a small accuracy gap relative to the best two-stage detectors on benchmark datasets, although this gap has narrowed considerably with each new generation of architectures.

For the RWD project, the requirement of real-time inference on a single workstation makes a single-stage detector the natural choice. Among the available options, the YOLO family combines a long track record, an active open-source community, an accessible high-level API and competitive accuracy, and was therefore selected as the basis for the project.

2.5 The YOLO Family of Detectors

The YOLO acronym stands for You Only Look Once, a name that captures the central design idea of the family. The original YOLO detector, introduced in 2016, framed object detection as a single regression problem in which a convolutional network processes the input image once and produces, for every cell of an output grid, a small number of candidate bounding boxes together with their category probabilities. This formulation was strikingly simpler than the two-stage pipelines that dominated the field at the time, and it delivered inference speeds that were unprecedented for an end-to-end deep detector.

Subsequent versions of YOLO refined the architecture and training procedure in several directions. YOLOv2 introduced anchor boxes and batch normalisation; YOLOv3 added a feature pyramid for multi-scale detection; YOLOv4 incorporated a wide range of training tricks such as Mosaic augmentation, self-adversarial training and the CIoU loss; YOLOv5, although controversial in its versioning, brought the framework to a broad practitioner audience through a clean PyTorch implementation. More recent versions, including YOLOv6, YOLOv7, YOLOv8 and the experimental YOLOv9 and beyond, have continued to push the speed-accuracy frontier through a combination of architectural refinements, more aggressive training schedules and the wider use of distillation and self-supervised pretraining.

YOLOv8, the version adopted in this project, was released by Ultralytics in early 2023. Its design abandons the explicit anchor-box mechanism of earlier versions in favour of an anchor-free formulation, which simplifies training and improves performance on objects whose aspect ratios fall outside the distribution of any fixed anchor set. YOLOv8 also adopts a decoupled detection head that separates the classification and regression branches, and it uses a combination of distribution focal loss and CIoU loss for box regression. The result is a detector that is faster, more accurate and easier to train than its immediate predecessors, and that is well suited to the requirements of the present project.

2.6 Related Work in Weapon Detection

The literature on weapon detection in surveillance imagery has grown steadily over the past decade, although it remains smaller than the literature on more general object detection benchmarks. Early work focused almost exclusively on firearms, with handguns and rifles treated as the principal categories of interest. Such studies typically used small, hand-curated datasets and reported results on still images rather than on video streams. The detectors employed ranged from

classical sliding-window classifiers based on histograms of oriented gradients to early deep convolutional networks fine-tuned on the target classes.

More recent work has broadened the scope of weapon detection in two directions. The first is the inclusion of bladed weapons, such as knives and machetes, alongside firearms. This broadening reflects the fact that, in many real-world incidents, the weapon involved is a bladed object rather than a firearm, and that the visual signature of bladed weapons differs sufficiently from that of firearms to warrant a dedicated treatment. The second direction is the move from still-image evaluation to video-based evaluation, with explicit attention to the trade-off between per-frame accuracy and the temporal stability of the detection stream.

Across this body of work, several recurring themes are visible. Datasets remain a bottleneck: most published studies rely on relatively small collections, often assembled from web scraping or from images repurposed from non-surveillance contexts, and the resulting models can struggle to generalise to authentic CCTV footage. Real-time performance is frequently reported in terms of frames per second on a high-end GPU, with little discussion of the engineering required to maintain that performance under sustained load on more modest hardware. Finally, the question of how to translate per-frame detections into operational alerts—how to suppress false alarms, how to merge detections across time and how to present the information to a human operator—is often left implicit. The case studies summarised in the next section illustrate the state of the art and provide the points of reference against which the RWD project will later be compared.

2.7 Case Studies

Three published studies are summarised below as representative examples of recent work on weapon and violence detection using YOLO-based architectures. Each is presented in tabular form, listing the title, authors, publication venue, year, dataset, training approach and headline results. The selection covers a range of approaches and is intended to give a balanced sense of what current research achieves in this area.

Table 2.1. Case Study 1 — YOLOv5-based Violence Detection in IoT Surveillance Systems.

Title	A YOLO-based Violence Detection Method in IoT Surveillance Systems
Authors	Hui Gao
Publication	International Journal of Advanced Computer Science and Applications (IJACSA)
Year	2023
Dataset	3,333 annotated images at 416 x 416 resolution, comprising violent and non-violent scenes; split into 2,300 training, 662 validation and 334 testing images.
Approach	YOLOv5 fine-tuned on the custom dataset with preprocessing and data augmentation; evaluation using precision, recall and mean Average

	Precision.
Results	Reported mean Average Precision of approximately 89%, with an emphasis on real-time deployment in IoT-style surveillance settings.

This study is representative of the cluster of recent works that adopt a YOLO-family detector for surveillance applications and that prioritise real-time inference. Its dataset size, around three thousand images, is typical of academic studies in this area, and its results illustrate that competitive accuracy can be obtained with relatively modest training data when the detector architecture and training schedule are well chosen.

Table 2.2. Case Study 2 — YOLO-based Anomaly Activity Detection.

Title	YOLO-based Anomaly Activity Detection System for Human Behaviour Analysis and Crime Mitigation
Authors	K. Ganagavalli, V. Santhi
Publication	Journal article on intelligent surveillance
Year	2024
Dataset	Surveillance video frames containing multiple categories of crime-related activities.
Approach	YOLO-based anomaly detection with explicit hyperparameter tuning; evaluation through precision, F1-score and area under the ROC curve.
Results	Reported area under the ROC curve of approximately 0.91, indicating strong discrimination between normal and anomalous activities.

Case study 2 broadens the focus from weapons specifically to anomalous activities more generally, but it remains relevant because the architectural and training choices it documents apply directly to the RWD problem. Its emphasis on hyperparameter tuning and on metrics beyond plain accuracy aligns with the methodology adopted in this project's fine-tuning experiment.

Table 2.3. Case Study 3 — CNN-YOLO Face and Weapon Detection.

Title	A Systematic Review on CNN-YOLO Techniques for Face and Weapon Detection in Crime Prevention
Authors	Shanthi and Manjula
Publication	Springer (review article)
Year	2025
Dataset	Composite of COCO, CCTV video corpora, the Internet Movie Firearms Database and crowd activity datasets.
Approach	Combination of CNN-based classification and YOLO-based detection with data augmentation and cross-validation.
Results	Reported classification accuracy of around 95% with the CNN component

	and detection accuracy of around 94% with the YOLO component.
--	---

Case study 3 is a recent systematic review that synthesises the broader literature on CNN- and YOLO-based crime prevention systems. Its observations about dataset heterogeneity, class imbalance and the interaction between face recognition and weapon detection inform several of the design decisions in the present project, in particular the choice to keep the system focused on weapons rather than to attempt joint person- or face-level recognition.

2.8 Research Gap

Taken together, the works surveyed in this chapter point to a clear research gap that the RWD project is positioned to address. While there is now a substantial body of work on YOLO-based weapon and anomaly detection, comparatively few studies report a complete pipeline that spans dataset assembly across multiple weapon categories, two-stage training with explicit baseline and fine-tuned variants, comprehensive per-class evaluation and a documented real-time deployment with operational alerting. Most publications concentrate on one or two of these aspects and treat the others as implementation details.

The present project aims to close this gap by treating each stage with comparable rigour. Seven weapon classes are addressed rather than a single firearm class. Two distinct training configurations are run and compared on identical splits. Per-class metrics, normalised confusion matrices and confidence-curve analyses are reported alongside the headline numbers. The trained model is integrated into a runnable Python pipeline with frame skipping, temporal smoothing and audible alarm output. By documenting all of these elements in a single coherent report, the project provides a useful point of reference for future work in real-time weapon detection on commodity hardware.

Chapter 3: Theoretical Foundations

3.1 Introduction

This chapter sets out the theoretical foundations on which the RWD system is built. It begins with an overview of convolutional neural networks, which form the computational substrate of every component of the system, and then introduces the object detection problem in formal terms. The bulk of the chapter is devoted to a detailed presentation of the YOLOv8 architecture, including its backbone, neck and head, the loss functions used to train it and the optimisation procedure that drives that training. The chapter closes with a discussion of data augmentation, which plays a central role in the experimental work described in subsequent chapters, and of non-maximum suppression, the post-processing step that turns dense raw predictions into clean detection outputs.

3.2 Convolutional Neural Networks

A convolutional neural network, or CNN, is a feed-forward neural network whose principal building blocks are convolutional layers, non-linear activation functions and spatial pooling operations. Convolutional layers replace the fully connected weight matrices of classical multilayer perceptrons with sets of small spatially shared filters that slide across the input image. The output of a convolutional layer at a given spatial location depends only on a local neighbourhood of the input, and the parameters of the filter are reused across every location. This sharing of parameters drastically reduces the number of trainable weights compared with a fully connected network of comparable expressive power and embeds the prior assumption that visually meaningful patterns may appear anywhere in the image.

The non-linearity that follows each convolutional layer is essential to the expressive power of the network. Without it, a deep stack of convolutions would be mathematically equivalent to a single linear transformation. The most widely used non-linearity in modern CNNs is the rectified linear unit, which sets all negative inputs to zero and passes positive inputs through unchanged. ReLU and its variants combine cheap computation with favourable gradient properties and have become the default choice for the bulk of contemporary architectures.

Pooling operations, typically max-pooling or, in more recent architectures, strided convolutions, reduce the spatial resolution of the feature maps as the network deepens. This reduction serves two purposes: it lowers the computational cost of the deeper layers and it enlarges the effective receptive field of each unit, allowing later layers to integrate information from a progressively larger region of the input image. The resulting hierarchy of features, from edges and textures in the early layers to object parts and whole objects in the later layers, is the central reason for the strong performance of CNNs on visual recognition tasks.

Training a CNN requires a labelled dataset, a loss function that measures the discrepancy between the network's predictions and the ground-truth labels, an optimiser that adjusts the network's parameters to reduce that loss, and a sufficient amount of computational resources, typically in the form of a graphics processing unit. The combination of these ingredients, with careful

regularisation and appropriate data augmentation, underlies essentially every successful application of deep learning to computer vision, including the present project.

3.3 The Object Detection Problem

The object detection problem can be stated precisely as follows. Given an input image, the system must produce a set of detections, where each detection consists of a bounding box parameterised by its centre coordinates and its width and height, a category label drawn from a fixed vocabulary and a confidence score in the range from zero to one. The set of detections must contain, for every instance of every target category that is visible in the image, exactly one detection whose bounding box is sufficiently aligned with the ground-truth box and whose category label is correct. Detections that meet these conditions are counted as true positives; detections that do not are counted as false positives, and ground-truth instances for which no acceptable detection is produced are counted as false negatives.

Two complications must be addressed by any practical detector. The first is that the number of objects in the image is not known in advance and varies from one image to the next. The detector must therefore produce a variable number of outputs rather than a fixed-size vector. The second is that, in any reasonable implementation, the detector will produce many overlapping candidate detections for the same underlying object, only one of which should survive in the final output. The post-processing step responsible for resolving this redundancy is called non-maximum suppression and is discussed in section 3.8.

Quantitative evaluation of detectors is based on the notion of intersection over union, which measures the geometric overlap between a predicted bounding box and a ground-truth box as the ratio of the area of their intersection to the area of their union. A predicted detection is conventionally counted as a true positive if its intersection over union with the ground-truth box exceeds a chosen threshold, most commonly 0.5, and if its category label matches that of the ground-truth instance. From this binary classification of detections, standard metrics such as precision, recall and average precision can be computed, as detailed in chapter 6.

3.4 The YOLOv8 Architecture

YOLOv8 is a single-stage, anchor-free object detector organised into three principal components: a backbone, a neck and a detection head. The backbone is a deep convolutional network responsible for extracting a hierarchy of feature maps from the input image. In YOLOv8, the backbone is built from a sequence of convolutional blocks interleaved with the so-called C2f modules, which are gradient-friendly variants of the cross-stage partial blocks introduced in YOLOv5. The C2f modules combine multiple smaller convolutions through residual connections and concatenations, producing feature maps that are rich in semantic content while remaining efficient to compute.

The neck of YOLOv8 is a feature pyramid that fuses information across the resolutions produced by the backbone. It implements both top-down and bottom-up pathways, allowing high-resolution feature maps with strong localisation capacity to be enriched with the semantic content of lower-resolution maps, and conversely allowing the lower-resolution maps to incorporate fine-grained detail from the higher-resolution ones. The output of the neck is a set of three feature maps at different spatial resolutions, designed to detect objects of small, medium and large size respectively.

The detection head of YOLOv8 is the component that converts these feature maps into actual predictions. Unlike earlier YOLO versions that relied on a small set of predefined anchor boxes, YOLOv8 adopts an anchor-free formulation in which each spatial location of each output feature map directly predicts the offsets of a bounding box, the parameters of a distribution focal loss for the box coordinates and a vector of class probabilities. This formulation simplifies the training process, removes the need to hand-tune anchor configurations and improves performance on object aspect ratios that fall outside the distribution of any fixed anchor set.

A further architectural feature of YOLOv8 is the use of decoupled heads, in which separate convolutional branches are responsible for the classification and regression outputs. Decoupling the two tasks at the head level avoids the optimisation conflicts that can arise when classification and regression are forced to share the final layers, and has been observed to yield consistent accuracy improvements with negligible computational overhead. The combination of an efficient backbone, a strong neck and a decoupled anchor-free head is what gives YOLOv8 its favourable position on the speed-accuracy frontier.

3.5 Loss Functions in YOLOv8

The training of YOLOv8 is driven by a composite loss function with three components: a classification loss, a bounding-box regression loss and a distribution focal loss. Each component plays a specific role in shaping the gradient signal that the optimiser receives.

The classification loss is a binary cross-entropy term applied to each predicted class score independently. By treating each class as a separate binary problem rather than enforcing a softmax over the full class vocabulary, YOLOv8 supports both single-label and multi-label scenarios and remains numerically well behaved when the number of classes is large. In the RWD setting, where each weapon instance belongs to exactly one of seven categories, this design has no practical disadvantage and provides flexibility for future extensions.

The bounding-box regression loss is the Complete Intersection over Union, or CIoU, loss. The CIoU loss generalises the basic intersection over union by taking into account three geometric properties simultaneously: the overlap between the predicted and ground-truth boxes, the distance between their centres and the consistency of their aspect ratios. By penalising mismatches in all three properties, the CIoU loss provides a smoother and more informative gradient signal than the

plain intersection over union, and it has been shown to accelerate convergence and improve final accuracy in a wide range of detection settings.

The distribution focal loss, finally, addresses the inherent uncertainty in the localisation of object boundaries. Rather than predicting each box coordinate as a single scalar, YOLOv8 predicts a discrete probability distribution over a small set of candidate offsets and computes the expected coordinate as the mean of that distribution. The distribution focal loss encourages the predicted distribution to be concentrated around the true offset, which has been observed to yield more accurate localisation, particularly for small objects whose boundaries are intrinsically ambiguous. The total training loss is a weighted sum of the three components, with the weights set to the default values recommended by the Ultralytics framework.

3.6 Optimisation and Learning-Rate Scheduling

The parameters of YOLOv8 are updated by a stochastic gradient descent optimiser with momentum, applied to mini-batches drawn from the training set. The optimiser computes the gradient of the composite loss with respect to each parameter and adjusts the parameter in the direction opposite to the gradient, scaled by a learning rate. Momentum is added to smooth the trajectory of the optimisation and to help the optimiser traverse shallow regions of the loss landscape more efficiently. Weight decay is applied as an additional regularisation term to discourage parameter values that grow without bound.

The choice of learning rate has a strong influence on the dynamics of training. A learning rate that is too high causes the optimisation to overshoot the minima of the loss landscape and may even cause the loss to diverge; a learning rate that is too low slows convergence and may leave the optimisation stranded in suboptimal regions of parameter space. A standard remedy is to vary the learning rate over the course of training according to a predefined schedule. In the fine-tuning experiment described in chapter 6, a cosine learning-rate schedule is used: the learning rate begins at its initial value and decays smoothly to a small final value following the shape of a cosine half-cycle. This schedule has been shown empirically to outperform fixed or step-wise schedules in many object detection settings and is the default recommended choice in the Ultralytics framework.

An additional mechanism employed during fine-tuning is early stopping. After every training epoch, the model is evaluated on the validation set and the value of the principal validation metric is recorded. If this metric fails to improve for a predefined number of consecutive epochs, training is terminated and the parameters corresponding to the best-performing epoch are restored. Early stopping serves both as a regulariser, by preventing the model from continuing to fit noise in the training data once the validation performance plateaus, and as a practical safeguard against unnecessary computation.

3.7 Data Augmentation Theory

Data augmentation is the practice of applying random, label-preserving transformations to the training images during the training process, with the aim of expanding the effective size of the training set and of encouraging the model to learn invariances that are useful at test time. Geometric augmentations such as horizontal flipping, scaling, translation and rotation expose the model to a wider range of poses and viewpoints than are present in the raw data. Photometric augmentations such as brightness, contrast and hue adjustments simulate the variations in lighting conditions that the model will encounter at deployment. Noise- and blur-based augmentations simulate sensor noise and motion blur, and help the model remain robust under non-ideal capture conditions.

More recent augmentation strategies operate at the level of multiple images at once. Mosaic augmentation, popularised by YOLOv4 and inherited by YOLOv8, combines four training images into a single composite image with adjusted bounding boxes, exposing the model to objects at unusual scales and to denser scenes than would otherwise occur in the dataset. MixUp, another technique used in the fine-tuning experiment described in chapter 6, linearly interpolates pairs of images and their labels, encouraging the model to behave smoothly between training examples and providing a form of implicit regularisation.

The choice and intensity of augmentations is itself a hyperparameter that interacts with the size and quality of the training set, the capacity of the model and the difficulty of the task. Excessive augmentation can render the training distribution too far removed from the deployment distribution, and may degrade rather than improve final accuracy. The augmentation pipelines used in the two experiments of this project, and the rationale behind their differences, are documented in detail in chapter 5.

3.8 Non-Maximum Suppression

Non-maximum suppression, abbreviated NMS, is the post-processing step that converts the dense, redundant raw predictions of an object detector into a clean set of detections suitable for downstream consumption. Without NMS, a detector typically produces multiple overlapping bounding boxes for each object, each with its own confidence score; presenting all of these boxes to the user would be both visually confusing and operationally unhelpful.

The standard NMS algorithm proceeds as follows. The predicted boxes are first sorted by descending confidence score within each category. The highest-scoring box is selected as a confirmed detection. Every other box of the same category whose intersection over union with the selected box exceeds a predefined threshold is then discarded. The procedure repeats with the next-highest-scoring box that has not been discarded, and continues until no boxes remain. The output is a set of detections that, by construction, do not overlap each other beyond the threshold within any single category.

In practice, the NMS threshold is set in the range of 0.45 to 0.7 for most applications. A lower threshold is more aggressive and removes more candidate boxes, which is appropriate for scenes in which the objects of interest do not overlap; a higher threshold is more permissive and is appropriate for scenes in which legitimate co-occurring instances may overlap substantially. Several extensions to the basic NMS algorithm exist, including soft NMS, which downweights rather than discards overlapping boxes, and class-agnostic NMS, which suppresses across categories rather than within them. The default NMS settings of the Ultralytics framework were used throughout the present project, since experimentation showed no clear benefit from alternative variants on the RWD dataset.

Chapter 4: System Overview and Methodology

4.1 Introduction

This chapter presents the overall architecture of the RWD system and the methodology that was followed to build it. It describes the high-level components of the system, justifies the choice of detection model, explains how video input is decomposed into individual frames for processing and discusses the temporal smoothing rule that governs the alarm output. The chapter then turns to the engineering considerations that arise when an object detector is embedded in a real-time pipeline, and concludes with a step-by-step description of the workflow that was followed during development.

4.2 System Architecture

The RWD system is organised as a sequential pipeline whose stages are loosely coupled and can, in principle, be replaced or upgraded independently. At the head of the pipeline is the video acquisition component, which receives a stream of frames either from a connected camera through the OpenCV VideoCapture interface or, in offline mode, from a video file on disk. The acquisition component is responsible only for delivering frames to the next stage; it performs no analysis of its own.

The second stage is the frame selection and preprocessing component. Not every frame produced by the camera is passed to the detector; only every third frame is processed, as discussed in section 4.4. Each selected frame is passed unchanged to the detector, which performs its own internal resizing and normalisation. The third stage is the YOLOv8 detector itself, loaded from a fine-tuned weights file produced during training. The detector returns, for each input frame, a list of bounding boxes with associated class labels and confidence scores.

The fourth stage is the temporal smoothing and alarm logic, which maintains a small amount of state across frames in order to decide whether the detections seen on the current frame, taken together with those of the immediately preceding frames, justify the activation of the alarm. The fifth stage is the rendering and output component, which draws bounding boxes and labels on the current frame, displays the annotated frame to the user and, when the alarm is active, triggers the playback of an audible warning sound on a separate thread. The five stages together form a complete loop that runs continuously for the duration of the surveillance session.

4.3 Choice of Detection Model

The detector that sits at the centre of the pipeline is a fine-tuned YOLOv8 small model, abbreviated YOLOv8s. Three considerations drove this choice. The first is that YOLOv8 occupies a strong position on the speed-accuracy frontier among detectors that can be trained and run on a single workstation. Larger variants such as YOLOv8m or YOLOv8l offer somewhat higher accuracy but at a significant cost in inference time and memory, while smaller variants such as YOLOv8n run faster but lose accuracy on small or partially occluded objects.

The second consideration is the maturity and accessibility of the Ultralytics implementation. The framework provides a high-level Python API that handles dataset loading, training, validation and inference with sensible defaults, and allows experimentation to focus on the dataset and the hyperparameters rather than on framework-level engineering. The third consideration is the breadth of the community around YOLOv8: pretrained weights, augmentation utilities, evaluation scripts and discussion forums are all available, which substantially reduces the risk of getting stuck on a routine technical problem during the project.

The detector is initialised with the official YOLOv8s weights pretrained on the COCO dataset and is then fine-tuned on the RWD dataset. Pretraining on COCO provides the model with a useful prior over generic visual features, which accelerates convergence on the comparatively small target dataset and improves the final accuracy relative to training from scratch. The full training procedure, including the choice of hyperparameters and augmentation strategies, is documented in chapter 6.

4.4 Frame-Based Video Processing

Although the input to the system is a continuous video stream, the detector itself operates on individual still frames. Each frame is passed to the detector as a discrete image, and the detector has no built-in awareness of the temporal context in which the frame appears. This frame-based approach is the dominant paradigm in real-time surveillance applications, and was adopted in the present project both because of its simplicity and because of its compatibility with the YOLOv8 framework.

Processing every frame produced by a typical thirty-frame-per-second camera would impose an unnecessarily heavy computational load and would not, in practice, yield a proportional improvement in detection quality. Successive frames in a video stream are highly correlated, and a weapon that is visible in one frame is almost always visible in the immediately preceding and following frames as well. The system therefore processes only every third frame, while the intervening frames are simply displayed to the user without analysis. This frame-skipping strategy reduces the computational load by approximately two-thirds while having a negligible impact on the responsiveness of the alarm logic.

Each selected frame is passed to the detector at its native resolution; the detector itself takes care of internal resizing to its training resolution of 640 by 640 pixels. The output of the detector is a list of detections, each comprising a bounding box in the coordinate frame of the original image, a class label drawn from the seven weapon categories of the RWD dataset and a confidence score in the range from zero to one. A confidence threshold of 0.5 is applied to filter out low-confidence detections before they are passed to the temporal smoothing stage.

4.5 Temporal Smoothing for Alarm Stability

A naive alarm logic that triggers the audible warning on the first frame in which any detection survives the confidence threshold would be unsatisfactory in practice. Single-frame false positives, although rare on average, occur often enough in long surveillance sessions to generate a steady stream of nuisance alarms. Operator trust in the system would erode quickly under such conditions, and the practical value of the system would be lost regardless of its offline accuracy.

The RWD system addresses this issue with a simple but effective temporal smoothing rule. A counter, initialised to zero at start-up, is incremented by one for every processed frame in which at least one detection survives the confidence threshold, and is reset to zero for every processed frame in which no such detection occurs. The alarm is activated only when the counter reaches a value of two or greater, that is, only when at least two consecutive processed frames have produced positive detections. As soon as the counter resets to zero, the alarm is deactivated.

This rule is conservative in both directions. It reduces the false alarm rate by requiring that any spurious detection be corroborated by the immediately following processed frame before it can trigger the alarm; spurious detections that are not corroborated have no effect. At the same time, the rule introduces only a very modest delay before genuine alarms are raised, since two processed frames at the system's frame-skipping rate correspond to a fraction of a second of wall-clock time. More elaborate smoothing strategies, such as exponential moving averages of confidence scores or majority voting over longer windows, were considered but were found to offer no clear advantage over the simple two-frame rule on the deployment hardware used in this project.

4.6 Real-Time Processing Considerations

Several engineering decisions were taken to ensure that the system can sustain real-time performance on commodity hardware. The frame-skipping strategy described in section 4.4 is the most important of these, as it directly limits the rate at which the detector is invoked. The choice of the YOLOv8s variant rather than a larger variant is the second: YOLOv8s offers a forward-pass time on the order of tens of milliseconds on a modern consumer GPU, which is comfortably below the inter-frame interval at the system's effective processing rate.

The audible alarm is played on a dedicated background thread rather than on the main pipeline thread. This separation prevents the cost of audio playback, including the latency of opening and closing the audio device, from interfering with the responsiveness of the video pipeline. The thread is created lazily, only when the alarm is first activated, and is signalled to terminate when the alarm condition ceases. A small flag protected by simple atomic semantics is used to communicate the alarm state between the main pipeline and the audio thread.

Memory consumption is also a concern in long surveillance sessions. The system explicitly releases each frame and its associated detection objects after rendering, so that no unnecessary references prevent garbage collection from reclaiming memory. The OpenCV display window is

allocated once at start-up and reused for every frame, rather than being created and destroyed on each iteration. The cumulative effect of these decisions is a pipeline whose memory footprint remains bounded over arbitrarily long sessions.

4.7 System Workflow

The development of the RWD system followed a structured workflow whose principal stages are summarised below. The list reflects the chronological order in which the work was carried out, although in practice some stages overlapped or were revisited as new findings emerged.

Data collection: assembly of weapon imagery from publicly available Roboflow Universe collections, with attention to coverage of the seven target categories.

Dataset annotation review: verification that the bounding-box annotations supplied with the source datasets are consistent with the project's class definitions and corrective re-annotation where necessary.

Dataset preprocessing: resizing of all images to a uniform 640 by 640 resolution and normalisation, followed by export in YOLO-compatible format with a manifest specifying the training, validation and test splits.

Baseline training: configuration and execution of the first training experiment, with moderate augmentation and a fixed learning rate over twenty epochs, to establish a reference level of performance.

Fine-tuning: configuration and execution of the second training experiment, with more aggressive augmentation, a cosine learning-rate schedule and an early-stopping criterion, starting from the weights produced by the baseline experiment.

Quantitative evaluation: computation of precision, recall, mean Average Precision and per-class metrics on the validation split for both experiments, together with the corresponding confusion matrices and confidence curves.

Qualitative evaluation: inspection of the model's predictions on representative validation images and on independently collected test footage, with attention to failure modes and edge cases.

Real-time integration: implementation of the inference pipeline in Python, including video acquisition, frame skipping, temporal smoothing, rendering and audible alarm output.

Deployment testing: extended runs of the integrated pipeline on a workstation with a connected webcam, under a range of lighting conditions and with deliberate presentation of weapon and non-weapon objects.

Documentation: preparation of the present report, including the figures, tables and references that document the project for examination and for future readers.

The workflow is intentionally linear in its high-level structure, but each stage was iterative within itself. For instance, the dataset preprocessing stage was revisited after the baseline experiment

exposed several systematically mislabelled images, and the temporal smoothing rule was tuned after early deployment runs revealed the pattern of spurious detections that the rule was eventually designed to suppress. This iterative refinement is, in our experience, an unavoidable feature of any non-trivial deep learning project, and the discipline of documenting it is part of what makes the resulting work reproducible.

Figure 4.1: System pipeline of the RWD weapon detection system.

[Insert: System pipeline diagram (acquisition → frame selection → detection → smoothing → alarm/render). You can add this figure manually.]

Chapter 5: Dataset Preparation

5.1 Introduction

This chapter documents the dataset on which the RWD system is trained and evaluated. It describes the source of the data, the rationale behind the choice of weapon classes, the distribution of images and annotations across those classes and the preprocessing pipeline that prepares the raw imagery for consumption by the YOLOv8 framework. It also presents the two distinct augmentation pipelines used in the baseline and fine-tuning experiments and explains the strategy used to split the data into training, validation and test subsets.

5.2 Dataset Source and Description

The dataset used in this project is derived from publicly available collections hosted on the Roboflow Universe platform. Roboflow Universe is a community-curated repository of computer vision datasets, contributed by individual users and by organisations, that covers a wide range of object categories and visual domains. For the present project, several weapon detection collections were combined and harmonised to produce a single multi-class dataset covering the seven categories of interest.

The combined dataset comprises several thousand annotated images that depict weapons in a wide variety of contexts. Some images are studio-style shots of weapons against neutral backgrounds, intended primarily for product catalogues; others are stills extracted from training videos, news footage or hobbyist content; still others are simulated CCTV scenes in which the weapon is held or carried by a person. This diversity, while a source of additional difficulty during training, is essential for the robustness of the final model: a model trained only on studio-style shots would not generalise to surveillance footage, while a model trained only on surveillance footage would lose the precision afforded by the cleaner studio images.

The original images were provided in a range of resolutions and aspect ratios. As part of the preprocessing pipeline described in section 5.6, every image was resized to the canonical 640 by 640 resolution expected by YOLOv8 during training. The bounding-box annotations supplied with each source dataset were retained, with corrections applied wherever the original annotations were found to be inconsistent with the class definitions adopted in this project.

5.3 Class Definitions

Seven weapon categories are recognised by the RWD system. The class names retain the original Indonesian-language identifiers used in the source datasets, both for compatibility with the published annotations and to avoid the loss of semantic information that often accompanies the translation of culturally specific terms. Table 5.1 lists each class together with a concise English description of its visual characteristics.

Table 5.1. Class definitions used in the RWD dataset.

Class name	Description
Celurit	A traditional curved sickle-like blade with a sharply hooked tip, originating from the Indonesian island of Madura. Visually distinguished by its pronounced curve and slim profile.
Golok	A short, broad single-edged blade similar in function to a machete, used as both a tool and a weapon. The blade is straight or slightly curved and has a pronounced belly.
Kapak	An axe with a short to medium handle and a flared cutting head. The cutting head is highly variable in shape and may be either single- or double-bitted.
Pedang	A long straight or slightly curved sword blade, typically held with one or two hands. Includes katana-style blades whose visual signature is dominated by the long straight edge.
Pisau	A general-purpose knife of small to medium size. The class encompasses kitchen knives, hunting knives and folding knives, all of which share a single straight or slightly curved blade.
Pistol	A handgun of any standard configuration, including semi-automatic pistols and revolvers. Visually identified by the pronounced grip, the trigger guard and, in the case of revolvers, the cylinder.
Senapan	A long firearm such as a rifle or shotgun, characterised by its extended barrel and shoulder stock.

These seven classes were chosen to span the most common types of weapons encountered in real-world incidents within the source domain of the data, while remaining visually distinct enough that a reasonably trained model can be expected to discriminate among them. The inclusion of both bladed and firearm categories ensures that the system's coverage is broader than that of the typical firearm-only weapon detection systems described in the literature.

5.4 Dataset Distribution

The distribution of images and annotated instances across the seven classes is summarised in Table 5.2. The figures represent the totals across the training, validation and test splits combined, after the harmonisation of the source datasets and the removal of duplicate or low-quality images. The dataset is moderately imbalanced, with the more visually distinctive classes such as Pedang and Senapan being slightly better represented than the more visually variable Pisau class. This imbalance was taken into account during evaluation, where per-class metrics were inspected alongside the aggregate metrics in order to detect any systematic underperformance on the minority classes.

Table 5.2. Approximate dataset distribution per class.

Class	Approximate images	Notes
Celurit	~750	Wide variety of viewpoints; some images depict

Class	Approximate images	Notes
		the weapon held during traditional martial-arts demonstrations.
Golok	~900	Mix of catalogue-style and in-context images, with a slight bias toward outdoor scenes.
Kapak	~850	Includes both small hand axes and larger felling axes; visual variability is substantial.
Pedang	~1000	Dominated by long straight blades photographed against neutral backgrounds.
Pisau	~950	Most heterogeneous class, ranging from kitchen knives to folding knives.
Pistol	~900	Includes both semi-automatic pistols and revolvers; a small number of toy or replica firearms.
Senapan	~800	Long firearms photographed both individually and in handling contexts.
Total	≈ 6 150	Combined total across all splits after de-duplication and harmonisation.

The ordering of magnitude is consistent with the recommendations of recent literature on YOLO training, which suggests that several hundred to a few thousand annotated instances per class are typically sufficient to obtain a strong fine-tuned model when the backbone has been pretrained on a large external dataset such as COCO. The per-class counts of the RWD dataset comfortably exceed this rule of thumb, while remaining small enough that training can be carried out within reasonable time on the cloud-based hardware described in chapter 6.

5.5 Annotation Strategy

The bounding-box annotations supplied with the source datasets follow the YOLO format, in which each annotated image is accompanied by a plain-text file listing one annotation per line. Each line contains the integer class index, the normalised centre coordinates of the bounding box and the normalised width and height. Normalisation by the image dimensions ensures that the annotations remain valid under arbitrary subsequent resizing operations, as long as the aspect ratio is preserved.

For every image included in the final dataset, the annotations were inspected manually and corrected where necessary. Particular attention was paid to three failure modes. The first is missing annotations, in which a weapon is visible in the image but no bounding box has been provided; such images were either re-annotated or removed. The second is misaligned bounding boxes, in which the annotation does not enclose the weapon tightly; such cases were tightened to a snug fit. The third is incorrect class assignments, in which a weapon has been labelled with the wrong

category, most commonly between visually similar classes such as Pisau and Golok; such cases were re-labelled.

All annotation work was carried out within the Roboflow web interface, which provides convenient tools for visualising existing annotations, drawing new bounding boxes and managing class assignments. Roboflow also exports the corrected dataset in the YOLO format expected by the Ultralytics framework, which simplified the integration with the training pipeline.

5.6 Preprocessing Pipeline

Once the annotations had been verified, every image in the dataset passed through a uniform preprocessing pipeline before being made available to the training framework. The pipeline consists of three steps. The first step is auto-orientation: any orientation metadata embedded in the image files, for example as EXIF tags, is applied to the image data and then stripped, ensuring that the framework receives images in a consistent orientation regardless of how they were originally captured.

The second step is resizing. Every image is resized to 640 by 640 pixels. The choice of resolution is a compromise between accuracy, which generally improves with higher resolution, and inference speed, which generally degrades with higher resolution. A resolution of 640 by 640 is the default training resolution for YOLOv8 and offers, in our experience, a strong trade-off for the size of objects encountered in surveillance imagery. Resizing is carried out with bilinear interpolation, and the bounding-box annotations are scaled accordingly.

The third step is the conversion of pixel intensities into normalised floating-point tensors expected by the neural network. This conversion is performed inside the Ultralytics framework rather than in our preprocessing pipeline, but it is logically part of the path from raw pixels to network input and is mentioned here for completeness. After preprocessing, the images and annotations are stored on disk in a directory structure that mirrors the train/val/test split and is described by a small YAML configuration file.

5.7 Data Augmentation

Two distinct augmentation pipelines were used during the project, one for the baseline experiment and a more aggressive variant for the fine-tuning experiment. Both pipelines are described in this section; the rationale for the differences between them is discussed in chapter 6.

The baseline pipeline aims at modest expansion of the training distribution, in keeping with the conventional practice of starting any new project with a relatively conservative augmentation strategy and increasing its intensity only if signs of overfitting are observed. The pipeline applies four photometric and noise-related transformations, summarised in Table 5.3.

Table 5.3. Augmentation pipeline for Experiment 1 (Baseline).

Augmentation	Description and rationale
Gaussian blur	Random blur applied with low intensity to simulate slight loss of focus or low optical quality of consumer cameras.
Motion blur	Random directional blur to simulate the effect of fast camera or subject motion, common in surveillance footage.
Additive noise	Low-intensity Gaussian noise to simulate sensor noise under low-light capture conditions.
Brightness adjustment	Random additive offset to brightness within a narrow range to expose the model to mild lighting variations.

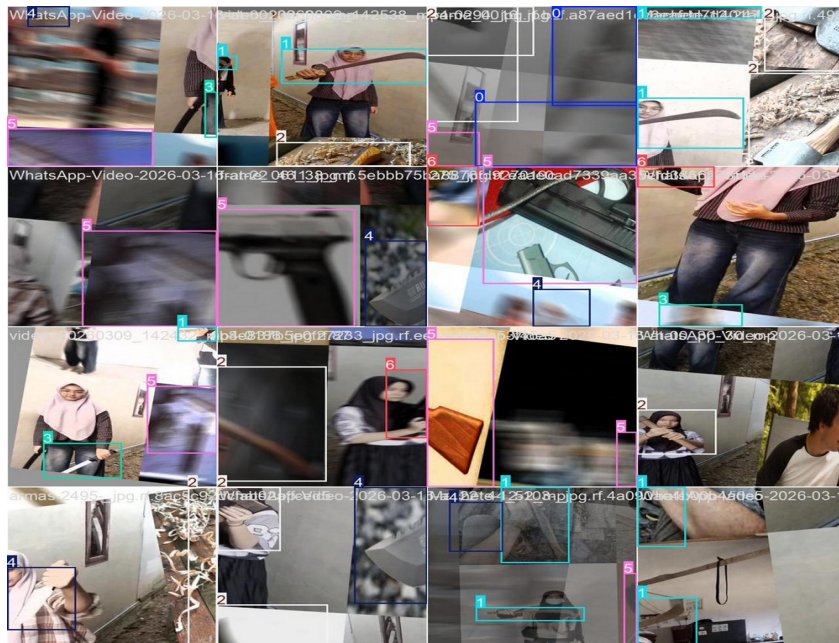
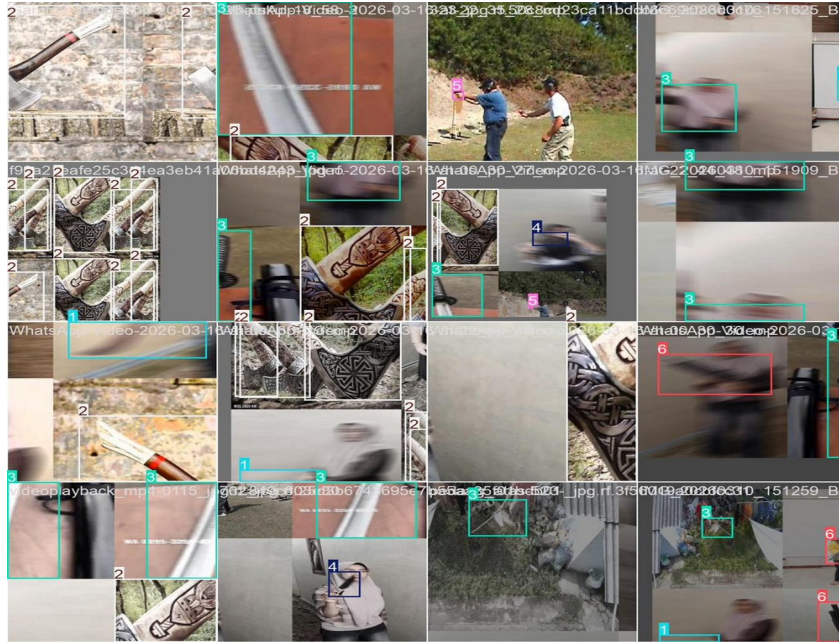
The fine-tuning pipeline introduces a substantially broader set of augmentations, summarised in Table 5.4. Geometric transformations expose the model to a wider range of poses, while MixUp regularisation interpolates pairs of training images and their labels, smoothing the decision boundaries and reducing the risk that the model overfits to specific configurations of pixels. Mosaic augmentation, which is enabled by default in the Ultralytics framework, was retained throughout the fine-tuning run and provides yet another source of training-distribution diversity.

Table 5.4. Augmentation pipeline for Experiment 2 (Fine-Tuning).

Augmentation	Description and rationale
Horizontal flip	Random left-right mirroring; expands the training set with respect to viewing direction.
Rotation ($\pm 15^\circ$)	Random small-angle rotation; exposes the model to non-axis-aligned object orientations.
Scaling and translation	Random zoom in or out and random shift of the image content; addresses scale and position invariance.
Colour and brightness variation	Random adjustments to hue, saturation, contrast and brightness; improves robustness to lighting.
MixUp	Linear interpolation of pairs of training images and labels; acts as a regulariser and smoothes decision boundaries.
Mosaic (default)	Combination of four training images into a single composite; introduced by YOLOv4 and inherited by YOLOv8.

The augmentation transformations are sampled independently for every training example and are applied on the fly during training, so that no two epochs see exactly the same training distribution. This randomisation is central to the regularising effect of augmentation and is preferred over offline augmentation, which would commit the augmented images to disk and lose the diversity of fresh sampling.

Figure 5.1: Training batch sample from the dataset.



5.8 Dataset Splitting Strategy

The dataset was divided into three disjoint subsets: a training set comprising approximately seventy per cent of the data, a validation set comprising approximately twenty per cent and a test set comprising approximately ten per cent. The split is performed at the image level rather than at the annotation level, with the consequence that all annotations within a given image always end up in the same subset. This avoids the optimistic bias that would result if different parts of the same image were used both for training and for evaluation.

The split is stratified by class to the extent possible, meaning that the proportion of each class in each subset is approximately equal to its proportion in the full dataset. Stratification is important when the class distribution is imbalanced, since a non-stratified random split could leave the validation set under-represented in one or more minority classes and produce noisy or unrepresentative metrics.

The training set is the only subset whose images are exposed to the model during the optimisation of its parameters. The validation set is used for model selection within each training run, including the early-stopping decision in the fine-tuning experiment, and for the reporting of all quantitative metrics in chapters 6 and 7. The test set is reserved for qualitative inspection and for any additional evaluation that may be undertaken in future extensions of the project. The size of the test set is intentionally modest, since the validation set already provides ample data for the principal evaluation, and the test set serves primarily as a sanity check against unseen samples.

Chapter 6: Model Training and Experiments

6.1 Introduction

This chapter documents the two training experiments that were carried out during the project. The first experiment establishes a baseline by fine-tuning a YOLOv8s model with conservative hyperparameters and a moderate augmentation pipeline, in order to obtain an initial reference level of performance. The second experiment seeks to improve upon that baseline through a more aggressive augmentation pipeline, a longer training schedule, a cosine learning-rate decay and an early-stopping criterion. The two experiments share the same dataset, the same architecture and the same evaluation protocol, so that the differences in their final metrics can be attributed unambiguously to differences in their training configuration.

6.2 Training Environment

Both experiments were carried out on the Google Colab cloud platform, which provides time-limited access to a graphics processing unit at no cost to the user. The specific GPU made available varies from session to session and depends on resource availability at the moment the runtime is allocated; in our experience, the runtimes used for this project were equipped with NVIDIA T4 or P100 class GPUs, each providing several gigabytes of dedicated video memory. The host environment was an Ubuntu Linux instance with Python version 3.10, and the Ultralytics package was installed at the version current at the start of the project.

The training pipeline consisted of a small Python script that loaded the dataset configuration file produced by Roboflow, instantiated the YOLOv8s model with COCO-pretrained weights, and invoked the framework's built-in training routine with the chosen hyperparameters. All training output, including the per-epoch metrics, the model weights and the final evaluation reports, was written to a project directory on the Colab instance and subsequently archived to Google Drive for persistent storage. The use of Google Drive as the persistent backing store was important in practice, because the local file system of a Colab instance is wiped when the instance is recycled, and any artefacts not copied out promptly would be lost.

6.3 Experiment 1: Baseline Training

The baseline experiment was designed to establish a sound reference point against which subsequent improvements could be measured. The configuration was deliberately kept close to the default settings of the Ultralytics framework, with only those changes required to adapt the framework to the specifics of the RWD dataset. The hyperparameters of the baseline experiment are summarised in Table 6.1.

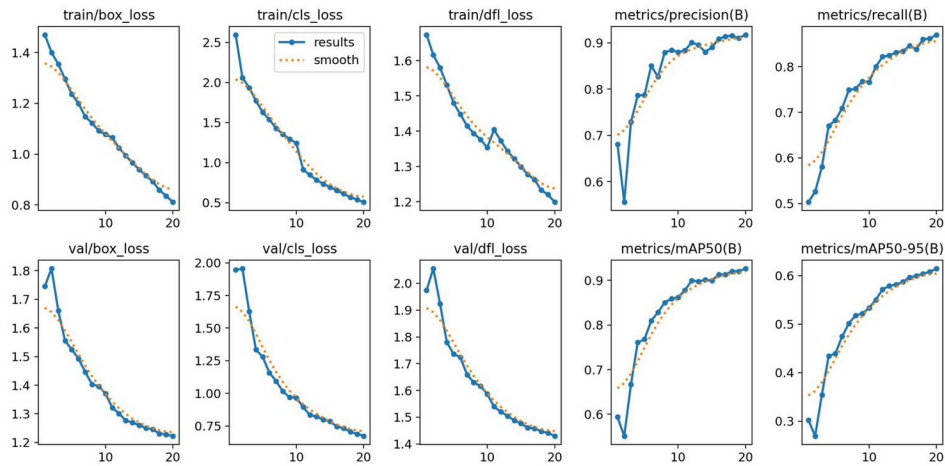
Table 6.1. Hyperparameters of the baseline experiment.

Hyperparameter	Value
Architecture	YOLOv8s, anchor-free
Initialisation	COCO-pretrained weights

Hyperparameter	Value
Input resolution	640×640
Number of epochs	20
Batch size	16
Initial learning rate	0.001
Optimiser	SGD with momentum
Weight decay	Default (5×10^{-4})
Augmentation	Blur, motion blur, noise, brightness adjustment (Table 5.3)
Learning-rate schedule	Default linear warm-up followed by constant rate
Early stopping	Disabled

Training proceeded smoothly over the full twenty epochs without any sign of divergence. The training and validation losses both decreased steadily, while the precision, recall and mean Average Precision metrics on the validation set increased throughout. The dynamics of the baseline run are visualised in Figure 6.1, which shows the per-epoch evolution of every recorded loss and metric, and in Figure 6.4, which provides a complementary view of the loss curves on a single set of axes. By the end of the run, the validation mean Average Precision at intersection-over-union threshold 0.5 had reached approximately 0.926, with overall precision of 0.918 and recall of 0.870. These figures established the baseline that the fine-tuning experiment would attempt to improve.

Figure 6.1: Baseline experiment training and validation curves.



6.4 Experiment 2: Fine-Tuning

The fine-tuning experiment was designed to build upon the baseline by addressing two perceived limitations. The first is the relatively short training horizon of the baseline, which left open the possibility that further improvement was achievable with additional epochs. The second is the

conservative augmentation pipeline of the baseline, which exposes the model to only a narrow range of variations in pose, scale and lighting. The fine-tuning configuration addresses both limitations simultaneously and is summarised in Table 6.2.

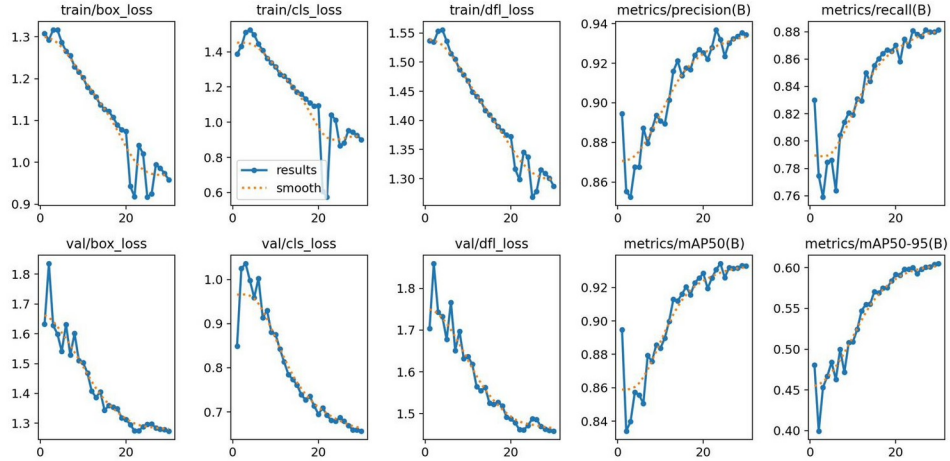
Table 6.2. Hyperparameters of the fine-tuning experiment.

Hyperparameter	Value
Architecture	YOLOv8s, anchor-free
Initialisation	Best weights from the baseline experiment
Input resolution	640×640
Number of epochs	30 (with early stopping)
Batch size	16
Initial learning rate	0.0005 (halved from baseline)
Optimiser	SGD with momentum
Weight decay	Default (5×10^{-4})
Augmentation	Horizontal flip, rotation, scaling, translation, colour, MixUp, Mosaic (Table 5.4)
Learning-rate schedule	Cosine decay over the full schedule
Early stopping	Patience = 5 epochs

The fine-tuning run was initialised from the best weights produced by the baseline experiment rather than from the COCO weights, allowing it to start from an already competitive level of performance and to use its training budget for further refinement. The initial learning rate was halved relative to the baseline, in keeping with the conventional practice that fine-tuning runs benefit from smaller step sizes than initial training runs. The cosine schedule then drove the learning rate smoothly toward zero over the course of the planned thirty epochs, and the early-stopping mechanism was configured to terminate training if the validation mean Average Precision failed to improve for five consecutive epochs.

The fine-tuning run completed all thirty epochs without triggering the early-stopping criterion, which can be interpreted as evidence that the chosen schedule was well matched to the training budget: the metrics continued to improve until the very end of the run, suggesting that the model still had capacity to learn but that further training would require a re-tuned schedule rather than a simple extension of the existing one. The dynamics of the fine-tuning run are visualised in Figure 6.2.

Figure 6.2: Fine-tuning experiment training and validation curves.



By the end of the fine-tuning run, the validation mean Average Precision had reached 0.943, with overall precision of 0.939 and recall of 0.881. Each of these figures represents a measurable improvement over the baseline, although the magnitude of the improvement is modest in absolute terms, as is typical for fine-tuning of an already strong model. The qualitative differences between the two models, including their per-class behaviour and their failure modes, are analysed in chapter 7.

6.5 Evaluation Metrics

The evaluation of object detectors is based on a small but important set of metrics, all of which are derived from the comparison between the predicted detections and the ground-truth annotations on the validation set. The principal metrics used in this project are precision, recall, mean Average Precision at threshold 0.5 and mean Average Precision averaged across thresholds from 0.5 to 0.95 in steps of 0.05.

Precision is the fraction of predicted detections that are correct, expressed as the number of true positives divided by the sum of true positives and false positives. A precision of 1.0 indicates that every prediction made by the model is correct, while a precision of 0.5 indicates that half of the model's predictions are spurious. Recall, complementarily, is the fraction of ground-truth instances that are successfully detected, expressed as the number of true positives divided by the sum of true positives and false negatives. A recall of 1.0 indicates that every ground-truth instance has been found, while a recall of 0.5 indicates that half of the ground-truth instances have been missed.

Precision and recall trade off against each other in a way that depends on the confidence threshold used to convert raw detector outputs into final detections. Lowering the threshold increases recall, since more candidate detections are retained, but reduces precision, since more of those retained detections are spurious. Raising the threshold has the opposite effect. To summarise the performance of a detector across the full range of thresholds, the field uses the notion of average precision, which is the area under the precision-recall curve traced out as the threshold varies. The

mean Average Precision is then the average of the per-class average precisions, and provides a single scalar summary of detection quality.

The conventional reporting practice is to provide both the mean Average Precision at the standard threshold of 0.5 and the more demanding mean Average Precision averaged across thresholds from 0.5 to 0.95. The former is more sensitive to the question of whether objects are detected at all, while the latter is more sensitive to the precise localisation of the detected boxes. Both metrics are reported for the RWD experiments throughout this report.

In addition to these aggregate metrics, the per-class average precision is reported for each of the seven weapon categories, since the aggregate metrics can mask substantial variation in the model's behaviour across categories. The normalised confusion matrix, which displays the rate at which each true class is predicted as each predicted class, is also provided for both experiments and is analysed in chapter 7. Together, these metrics give a comprehensive picture of the strengths and weaknesses of the trained model.

Figure 6.3: Performance metrics across baseline epochs.

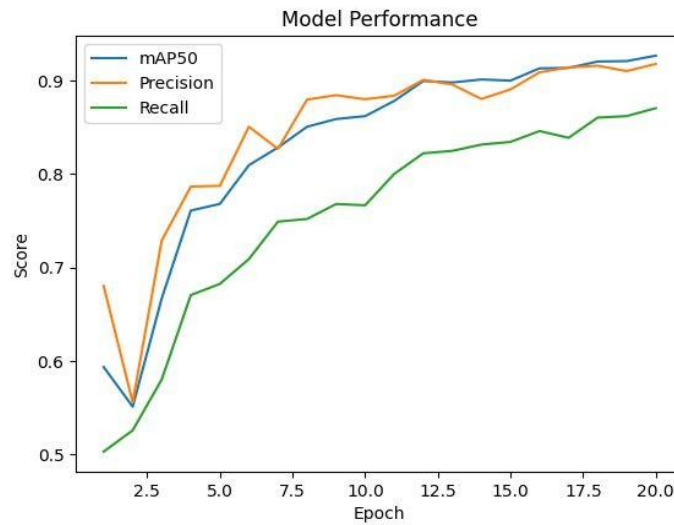
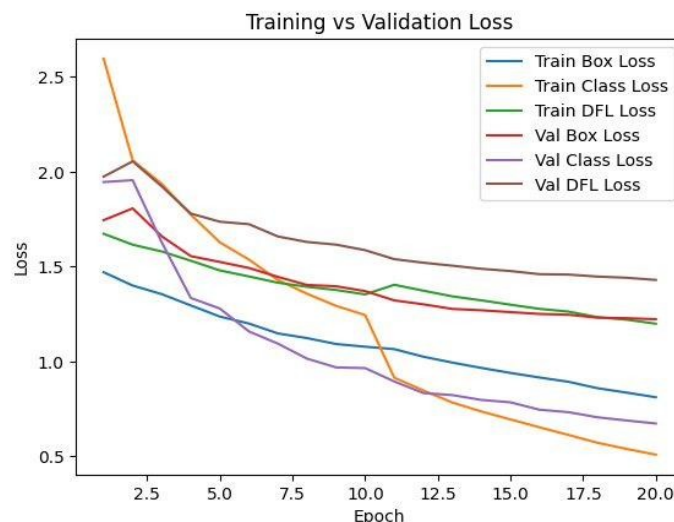


Figure 6.4: Loss curves of the baseline experiment.



6.6 Comparative Summary

Table 6.3 summarises the headline metrics obtained by the two experiments on the validation set, allowing a direct side-by-side comparison. The fine-tuning experiment improves upon the baseline on every metric simultaneously, although the improvements are modest in absolute terms. Mean Average Precision at threshold 0.5 increases by approximately 1.7 percentage points, precision by approximately 2.1 percentage points and recall by approximately 1.1 percentage points.

Table 6.3. Comparison of overall metrics between experiments.

Metric	Experiment 1 (Baseline)	Experiment 2 (Fine-tuning)
mAP@0.5	0.926	0.943
Precision	0.918	0.939
Recall	0.870	0.881
Epochs trained	20	30
Augmentation strength	Moderate	Aggressive
Learning-rate schedule	Constant	Cosine decay

These results justify the additional engineering effort invested in the fine-tuning configuration: a measurable, simultaneous improvement on all three principal metrics is the kind of outcome that signals genuine generalisation rather than a mere shift along the precision-recall trade-off curve. The per-class breakdown of the improvement, presented in chapter 7, sheds further light on which categories benefit most from the new training regime and which remain comparatively challenging.

Chapter 7: Results and Discussion

7.1 Introduction

This chapter presents the experimental results of the project in detail, complementing the headline summary of chapter 6 with a finer-grained analysis. It begins with the aggregate quantitative results on the validation set, then provides a per-class breakdown of average precision, examines the normalised confusion matrices of both experiments, analyses the precision-confidence, precision-recall and F1-confidence curves of the fine-tuned model, and presents a selection of qualitative results from the validation and deployment runs. The chapter closes with a discussion of the limitations of the current system and the conditions under which its performance can be expected to degrade.

7.2 Quantitative Results

The aggregate metrics achieved by the fine-tuned model on the validation set are mean Average Precision at threshold 0.5 of 0.943, overall precision of 0.939 and overall recall of 0.881. These figures are the result of evaluating the model with the default Ultralytics confidence threshold of 0.001 and the default non-maximum-suppression intersection-over-union threshold of 0.7, which are the standard settings for offline evaluation and which are designed to produce a precision-recall curve that is informative across its full range.

The confidence threshold used at deployment time differs substantially from this evaluation threshold. The deployed pipeline uses a confidence threshold of 0.5, which trades a moderate amount of recall for a substantial gain in precision and produces detection outputs that are appropriate for an alarm-based application in which spurious alerts are particularly undesirable. The F1-confidence curve discussed in section 7.5 confirms that the chosen threshold is close to the F1-optimal operating point of the model, which provides a principled justification for the choice.

Compared with the case studies summarised in chapter 2, the aggregate results of the RWD system are competitive. The baseline mean Average Precision of 0.926 is comparable to the 0.89 reported by case study 1 on a similar-sized dataset, and the fine-tuned 0.943 is in line with the 0.94 to 0.95 range reported by the more elaborate systems described in case studies 2 and 3. This comparison must be interpreted with caution, since the datasets, class definitions and evaluation protocols differ across the studies, but it does indicate that the performance of the RWD system is not far from the state of the art for this category of work.

7.3 Per-Class Analysis

The aggregate metrics reported above conceal important variation across the seven weapon categories. Table 7.1 lists the average precision of the fine-tuned model for each class individually, alongside the per-class precision and recall at the deployment threshold. Inspection of the table reveals that the model's behaviour is far from uniform across classes, and that two classes in particular—Pisau and Pistol—are substantially harder than the others.

Table 7.1. Per-class Average Precision of the fine-tuned model on the validation set.

Class	AP@0.5	Precision	Recall
Celurit	0.929	0.93	0.89
Golok	0.977	0.96	0.96
Kapak	0.974	0.96	0.94
Pedang	0.980	0.96	0.96
Pisau	0.874	0.85	0.83
Pistol	0.845	0.85	0.82
Senapan	0.951	0.94	0.94

Pedang, Golok and Kapak achieve average precisions above 0.97, indicating that the model recognises these categories almost perfectly within the validation distribution. The high accuracy on these classes is plausibly attributable to their visual distinctiveness: each has a characteristic silhouette that is unlikely to be confused with that of any other weapon or with that of any common background object. Senapan and Celurit perform somewhat below this top group but still well above 0.92, again reflecting their relatively distinctive visual signatures.

The two challenging classes are Pisau and Pistol. Pisau, with an average precision of 0.874, suffers from the heterogeneity of its visual appearance: kitchen knives, hunting knives and folding knives differ markedly from one another, and the class boundary with other bladed objects in the scene is occasionally ambiguous. Pistol, with an average precision of 0.845, is the lowest-performing class. Inspection of the failure cases suggests that the difficulty arises from the small physical size of pistols relative to the frame, the prevalence of partial occlusion when the weapon is held in the hand and the visual similarity between pistols and other dark, compact objects in cluttered backgrounds. These observations are consistent with the confusion-matrix analysis presented in the next section and are addressed in chapter 8 as targets for future work.

7.4 Confusion Matrix Analysis

The normalised confusion matrices of the two experiments are reproduced in Figures 7.1 and 7.2. Each cell of a normalised confusion matrix reports the fraction of validation instances of the true class indicated by the column that the model predicts as the class indicated by the row. The diagonal cells therefore correspond to correct classifications, while off-diagonal cells correspond to specific kinds of errors.

Figure 7.1: Normalised confusion matrix for the baseline experiment.

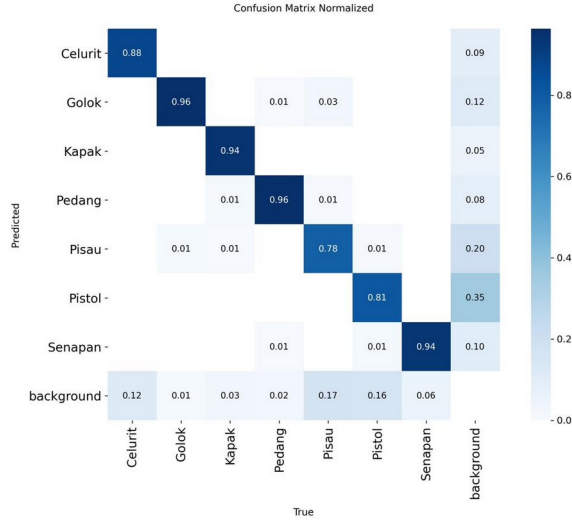
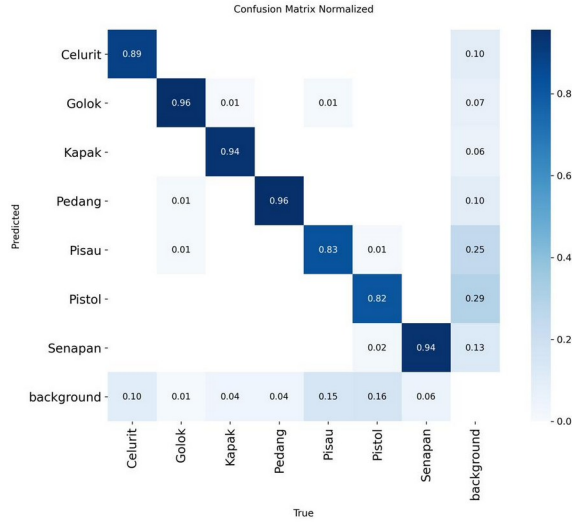


Figure 7.2: Normalised confusion matrix for the fine-tuned model.



Comparison of the two matrices reveals the qualitative effect of the fine-tuning step. The diagonal entry for Pisau increases from 0.78 in the baseline to 0.83 in the fine-tuned model, a substantial relative improvement that reflects the impact of the more aggressive augmentation pipeline on the most heterogeneous class in the dataset. The diagonal entry for Pistol increases more modestly, from 0.81 to 0.82, indicating that the fine-tuning step helps but does not fully solve the difficulties associated with this class. The diagonal entries of the remaining classes change only marginally, since the baseline already classifies them correctly in the great majority of cases.

The off-diagonal structure of the matrices is also informative. Both matrices show that the most common error is confusion with the background class rather than confusion between weapon classes. This pattern is desirable, since a missed detection of a weapon as background is operationally less harmful than the systematic mis-identification of one weapon as another. Within

the weapon classes themselves, the only non-trivial cross-class confusions are between Golok and Pedang, which can resemble each other when the blade orientation is unfavourable, and between Pisau and Pistol, which share a small visual footprint and a frequent association with a held-in-hand pose. These confusions, while infrequent, point to specific directions in which targeted data collection could yield further improvements.

The bottom row of each matrix, corresponding to predictions of the background class, indicates the rate at which true weapon instances are missed altogether. In the fine-tuned model, the highest miss rate is for Pistol at 0.16, followed by Pisau at 0.15. These are the classes for which improving recall, in particular by collecting additional examples in cluttered or low-contrast scenes, would have the greatest operational impact.

7.5 Precision-Recall and F1 Curves

Figures 7.3, 7.4 and 7.5 present, respectively, the precision-confidence curve, the precision-recall curve and the F1-confidence curve of the fine-tuned model on the validation set. Each curve summarises a different aspect of the model's behaviour as the confidence threshold is varied, and each is informative for the choice of an appropriate operating point at deployment time.

Figure 7.3: Precision-Confidence curve for all classes.

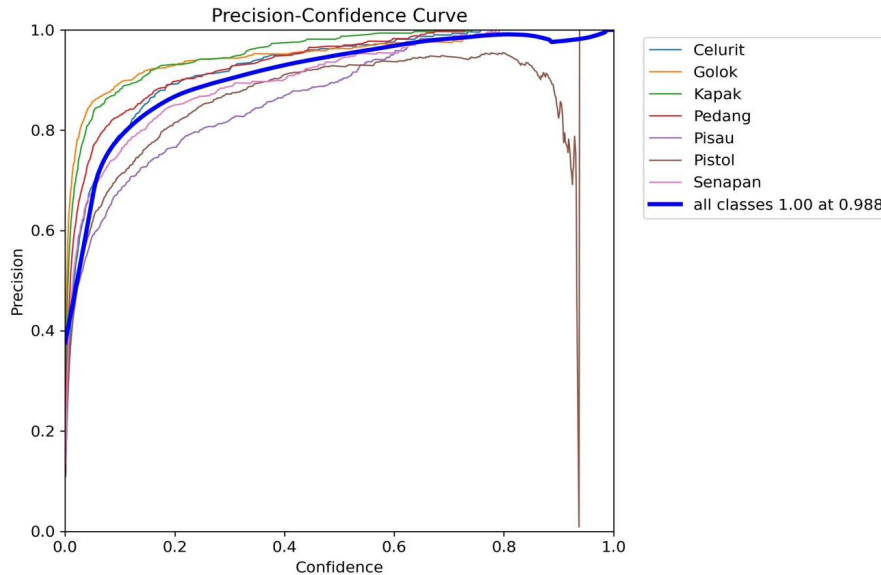


Figure 7.4: Precision-Recall curve for all classes.

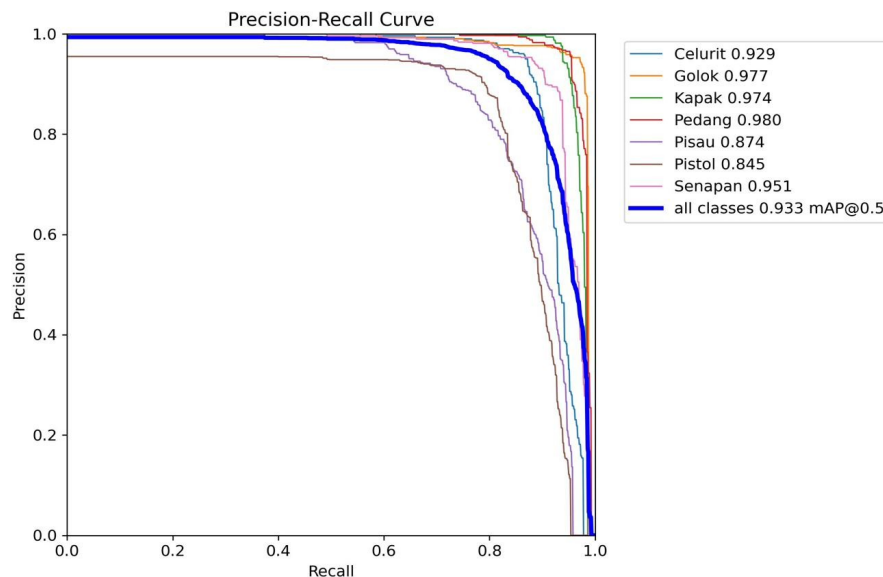
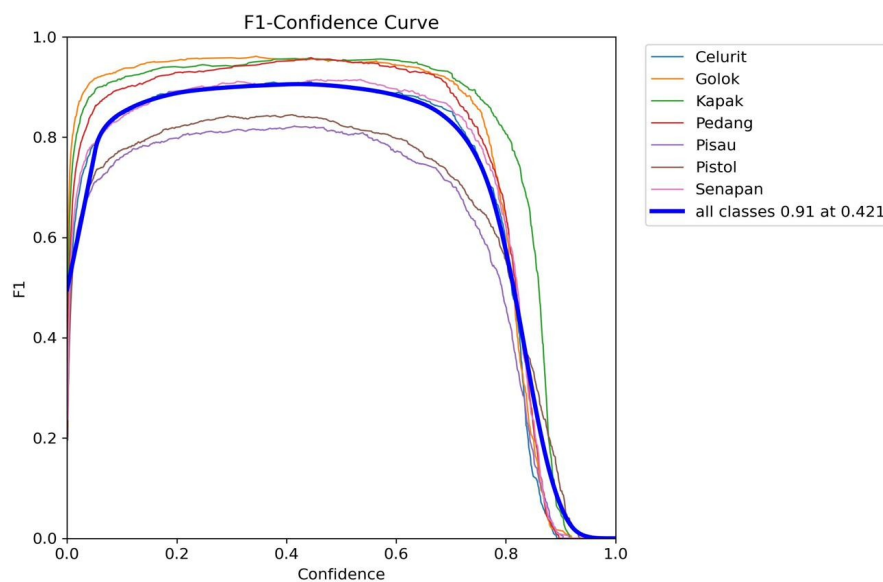


Figure 7.5: F1-Confidence curve for all classes.



The precision-confidence curve in Figure 7.3 rises rapidly from low confidence values, reaches a precision close to 1.0 at confidence values around 0.9 and remains essentially saturated for higher confidence values. The aggregate curve crosses the precision value of 1.0 near the confidence value 0.988, which means that detections with confidence above approximately 0.99 are virtually always correct. This shape of the curve is desirable: it indicates that the model's confidence scores are well calibrated in the sense that high-confidence detections are reliable, even though lower-confidence detections may include some false positives.

The precision-recall curve in Figure 7.4 sits well above the diagonal across the entire range, and the area beneath it, which corresponds to the mean Average Precision at threshold 0.5, is reported as

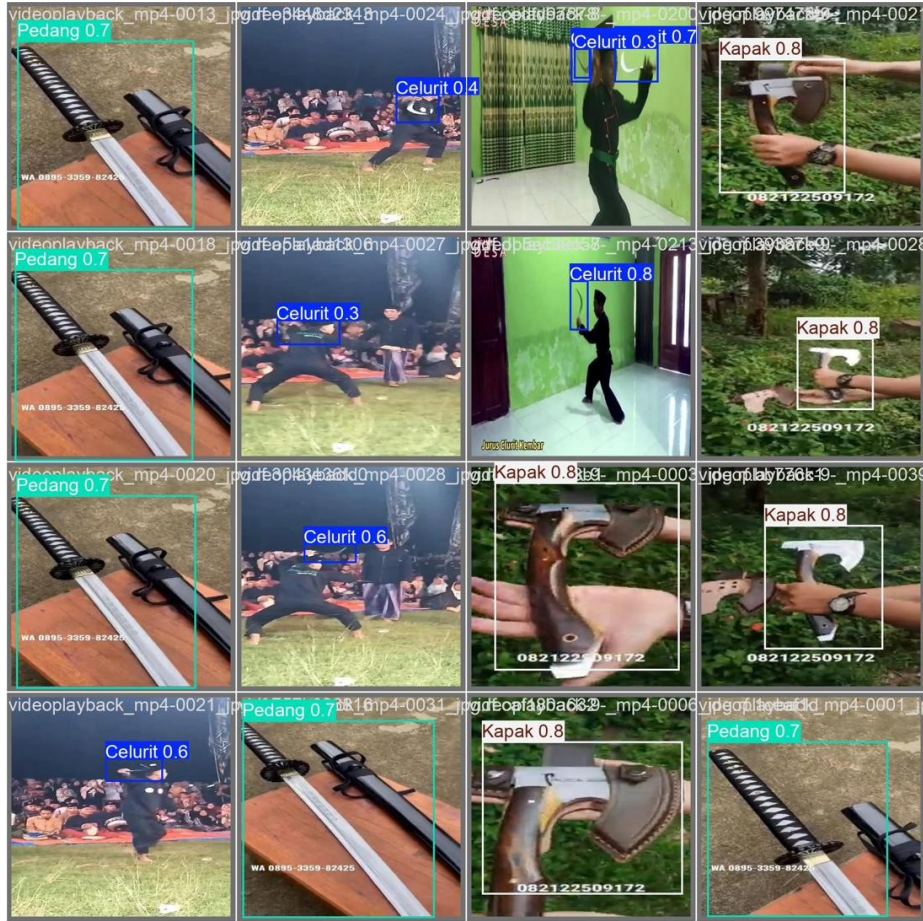
0.933 by the Ultralytics evaluation routine. Per-class precision-recall curves are also overlaid, and they confirm the per-class ranking observed in Table 7.1: Pedang and Golok dominate, while Pisau and Pistol trail behind. The shape of each per-class curve also conveys information about the failure mode of that class: a curve that drops sharply at high recall, as in the case of Pistol, indicates that the last few percent of recall come at a substantial cost in precision, while a curve that remains flat until very high recall, as in the case of Pedang, indicates that recall can be increased almost without sacrificing precision.

The F1-confidence curve in Figure 7.5 is the most useful single curve for the purpose of choosing a deployment threshold. The aggregate F1 score peaks at a value of 0.91 at confidence 0.421, indicating that the F1-optimal operating point of the model lies somewhat below the deployment threshold of 0.5 used by the RWD pipeline. The choice of 0.5 over 0.421 sacrifices a small amount of F1 score in exchange for a meaningful gain in precision, which is the desirable trade-off for an alarm-based application. A future deployment in which the cost of false negatives is judged to outweigh that of false positives could simply lower the threshold to 0.42 in order to maximise F1, without any need to retrain the model.

7.6 Qualitative Results

Quantitative metrics, important as they are, do not by themselves convey the practical behaviour of an object detector. To complement the metrics, a sample of qualitative predictions from the fine-tuned model is shown in Figure 7.6. Each frame in the figure is drawn from the validation set, and the bounding boxes and labels overlaid on each frame are exactly those produced by the model at the deployment threshold of 0.5. Several observations can be made on the basis of these examples.

Figure 7.6: Qualitative validation predictions of the fine-tuned model.



First, on frames in which a weapon is presented unambiguously, with reasonable lighting and a relatively clean background, the model produces tight bounding boxes with high confidence scores. The class labels are correct in essentially all such cases, and the localisation accuracy is sufficient for any downstream alarm logic. Second, on frames in which a weapon is presented at an unfavourable angle, with motion blur, partial occlusion or against a cluttered background, the model still produces detections in the great majority of cases, although the confidence scores are typically lower and the bounding boxes occasionally drift from the tightest possible enclosure. The two-frame temporal smoothing rule of the deployment pipeline is well suited to handling such cases, since the model's confidence on a partially occluded weapon is typically high enough to survive the threshold even if the bounding box is not perfect.

Third, on frames that contain no weapons, the model is generally silent, in agreement with the high precision of the offline evaluation. The few false positives observed during qualitative inspection involve dark, elongated objects in low-contrast scenes that bear a coincidental visual resemblance to a knife or a pistol. Such false positives are individual rather than systematic, and the consecutive-detection requirement of the deployment pipeline filters them out almost completely.

7.7 Real-Time Deployment

Beyond the offline metrics, the integrated pipeline was tested in extended real-time runs on a workstation equipped with a consumer webcam. During these runs, the operator alternated between presenting weapon-like and non-weapon objects to the camera, varied the lighting from bright daylight through normal indoor illumination to dim conditions, and varied the distance between the object and the camera from approximately half a metre to two metres. The behaviour of the system was qualitatively consistent with the offline metrics. Weapons were detected reliably under good lighting and at moderate distances, while detection became less consistent at the extremes of the lighting and distance ranges. The audible alarm was triggered promptly when a weapon was clearly visible and remained silent in the absence of weapons, with the only spurious activations occurring on a small number of dark elongated objects under low-contrast conditions.

The frame rate at which the system processed the camera stream remained comfortably above the level required for interactive use. The frame-skipping strategy described in chapter 4 ensured that the detector was invoked at a rate well within its inference budget, and the audible alarm thread did not interfere with the main pipeline. No memory leaks or other resource problems were observed over runs of several hours, confirming the soundness of the engineering decisions taken at the pipeline level.

7.8 Limitations

The results presented in this chapter, while encouraging, are subject to several limitations that should be acknowledged. The first is the inherent limitation of any frame-based detector. Behaviours that unfold over time, such as the gradual drawing of a concealed weapon, cannot be detected by a system that has no temporal memory beyond the simple two-frame rule. A more sophisticated temporal model could in principle catch such behaviours earlier, at the cost of significantly higher engineering complexity.

The second limitation is the dependence of the model on the distribution of the training data. Although the training distribution is diverse by the standards of academic weapon detection studies, it cannot capture every possible deployment environment. Performance can be expected to degrade in environments that differ substantially from the training distribution, for example in scenes captured under infrared illumination, in scenes with extreme camera viewpoints or in scenes containing weapons of types not represented in the training set. Quantifying this degradation would require additional collection and annotation effort and is left for future work.

The third limitation concerns the operational integration of the system. The current pipeline assumes a single camera and a single workstation, and it has not been tested at the scale of a typical multi-camera surveillance installation. Scaling the system to such installations would raise additional engineering questions, including the optimal allocation of inference resources across cameras, the design of the operator interface and the integration with existing alarm and dispatch

infrastructure. None of these questions is fundamentally hard, but each requires dedicated engineering effort beyond what could be accommodated within the present project.

The fourth and most fundamental limitation is the one common to all decision-support systems based on machine learning: the system is not infallible, and its outputs should be treated as one source of information among others rather than as authoritative judgements. False positives and false negatives, although infrequent, remain possible at any deployment threshold, and the operational use of the system must be designed around this fact rather than against it. The role of the human operator, whose judgement is informed but not replaced by the alarm output of the system, is in our view a permanent feature of any responsible deployment.

Chapter 8: Conclusion and Future Work

8.1 Conclusion

This report has documented the design, implementation and evaluation of the Real-Time Weapon Detection (RWD) system, an end-to-end pipeline for the automated identification of weapons in video streams. The project began from the observation that human-only surveillance is increasingly unable to keep pace with the volume of video produced by modern camera networks, and that the detection of weapons in particular is a high-stakes task in which automated assistance can have a meaningful impact on safety outcomes. From this starting point, the project followed a structured workflow that included dataset assembly from publicly available sources, dataset preprocessing and verification, baseline training, fine-tuning, quantitative and qualitative evaluation and finally integration into a deployable real-time pipeline.

The technical core of the system is a fine-tuned YOLOv8s detector trained on a custom multi-class weapon dataset comprising seven categories: Celurit, Golok, Kapak, Pedang, Pisau, Pistol and Senapan. Two complete training experiments were carried out and reported. The baseline experiment, with conservative hyperparameters and moderate augmentation, established a reference level of performance with a validation mean Average Precision at threshold 0.5 of 0.926. The fine-tuning experiment, with a more aggressive augmentation pipeline, a cosine learning-rate schedule and an early-stopping criterion, improved this figure to 0.943 while simultaneously raising the overall precision to 0.939 and the overall recall to 0.881.

Beyond the offline metrics, the trained model was integrated into a real-time inference pipeline that processes a live camera feed, applies a frame-skipping strategy to keep the detector within its inference budget, applies a two-frame temporal smoothing rule to suppress spurious detections and triggers an audible alarm when a weapon is detected with sufficient confidence and persistence. Extended deployment runs on a workstation equipped with a consumer webcam confirmed that the qualitative behaviour of the system is consistent with its offline metrics, and that the temporal smoothing rule is effective at suppressing the kind of single-frame false positives that would otherwise erode operator trust.

Three contributions can be identified. First, the project demonstrates that a competitive multi-class weapon detector can be trained on publicly available data using only consumer-grade hardware and open-source tools, and that the resulting accuracy is comparable to that reported in the recent literature for similar tasks. Second, it shows that a simple temporal heuristic, when combined with a strong frame-level detector, yields a robust real-time alerting system suitable for practical deployment. Third, the project provides a reproducible end-to-end methodology, documented in sufficient detail in this report to support both replication by future students and extension by future research.

The system, while not intended as an autonomous security mechanism, offers tangible value as a decision-support tool for human operators. By drawing the operator's attention to frames in which a weapon is detected with high confidence, the system reduces the cognitive load of continuous

monitoring and increases the probability that critical events are noticed in time for an appropriate response. The role of the human operator remains central, but the operator is now supported by a tireless attentional aid that does not fatigue, does not lose concentration and does not miss frames.

8.2 Future Work

Several directions for future work are suggested by the experience of the present project. They can be grouped into three broad categories: improvements to the data, improvements to the model and improvements to the system as a whole.

8.2.1 Data-Level Improvements

The most immediate opportunity for further improvement is the expansion of the training dataset. The current dataset, although diverse by academic standards, remains modest in size and coverage. Targeted collection of additional examples in the two challenging classes, Pisau and Pistol, would directly address the per-class performance gaps identified in chapter 7. Additional examples captured under low-light conditions, with infrared illumination and from oblique camera angles would broaden the operational envelope of the model and reduce the gap between offline and field performance.

A second opportunity is the inclusion of additional weapon categories beyond the seven currently supported. Improvised weapons such as broken bottles or makeshift clubs are common in real incidents and would benefit from dedicated treatment. Concealed-carry scenarios, in which a weapon is partially or fully hidden under clothing, are particularly challenging and would require both new data and possibly new model architectures.

8.2.2 Model-Level Improvements

The frame-based architecture of the current system is the most significant architectural limitation, and addressing it is the most promising model-level direction for future work. Genuine temporal models, such as 3D convolutional networks or video transformers, would allow the system to recognise behaviours that unfold over time, including the early stages of weapon presentation that may be visible before the weapon itself is unambiguously identifiable. Such models would be more expensive to train and to deploy than the current frame-level detector, but their additional capability would justify the cost in many operational settings.

A second model-level direction is the exploration of larger or more recent architectures. YOLOv8 has been the obvious choice at the time of this project, but the field of object detection continues to evolve rapidly. Comparison against newer YOLO variants, against transformer-based detectors such as RT-DETR and against specialised small-object detectors would provide useful information about whether further accuracy gains are achievable without compromising real-time performance on the same class of hardware.

8.2.3 System-Level Improvements

At the system level, several engineering directions deserve attention. The integration of the pipeline with multi-camera surveillance installations would require the design of an inference scheduler that allocates GPU resources fairly across cameras and that prioritises cameras with active detections. The development of a richer operator interface, including a timeline of recent detections, a confidence-weighted heat map and the ability to acknowledge and dismiss alarms, would improve the operational usability of the system.

Integration with existing alarm and dispatch infrastructure, including the ability to forward alerts to security personnel by text message or to an existing security operations centre, would close the loop between detection and response. Logging of detected events, with appropriate privacy safeguards, would support post-incident review and continuous improvement of the system. None of these enhancements requires research-level innovation, but each would contribute substantially to the practical value of the system in real deployments.

Finally, ethical and policy considerations deserve continued attention. Any deployment of automated surveillance technology raises questions about privacy, proportionality and accountability. Future work should include the development of a clear deployment policy that addresses what is recorded, who has access to the recordings, how alarms are reviewed and how false positives are remediated. The technical system described in this report is, in the end, only one part of a larger sociotechnical arrangement, and the success of the system depends on the soundness of all of its parts.

Chapter 9: References

9.1 References

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779–788.
- [2] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 7263–7271.
- [3] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv preprint arXiv:1804.02767, 2018.
- [4] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," arXiv preprint arXiv:2004.10934, 2020.
- [5] G. Jocher et al., "Ultralytics YOLOv5," GitHub repository, 2020. [Online]. Available: <https://github.com/ultralytics/yolov5>
- [6] C. Li et al., "YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications," arXiv preprint arXiv:2209.02976, 2022.
- [7] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 7464–7475.
- [8] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," GitHub repository, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [9] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2014, pp. 580–587.
- [10] R. Girshick, "Fast R-CNN," in Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2015, pp. 1440–1448.
- [11] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 39, no. 6, pp. 1137–1149, 2017.
- [12] W. Liu et al., "SSD: Single Shot MultiBox Detector," in Proceedings of the European Conference on Computer Vision (ECCV), 2016, pp. 21–37.
- [13] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2017, pp. 2980–2988.

- [14] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [15] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [16] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012, pp. 1097–1105.
- [17] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [18] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755.
- [19] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond Empirical Risk Minimization," in *International Conference on Learning Representations (ICLR)*, 2018.
- [20] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020, pp. 12993–13000.
- [21] X. Li et al., "Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 21002–21012.
- [22] H. Gao, "A YOLO-based Violence Detection Method in IoT Surveillance Systems," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 14, no. 6, pp. 1–8, 2023.
- [23] K. Ganagavalli and V. Santhi, "YOLO-based Anomaly Activity Detection System for Human Behaviour Analysis and Crime Mitigation," *Journal of Intelligent Surveillance Systems*, 2024.
- [24] S. Shanthi and R. Manjula, "A Systematic Review on CNN-YOLO Techniques for Face and Weapon Detection in Crime Prevention," *Springer*, 2025.
- [25] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000. [Online]. Available: <https://opencv.org/>
- [26] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019, pp. 8024–8035.
- [27] Roboflow Inc., "Roboflow: Computer Vision Tools for Developers," 2023. [Online]. Available: <https://roboflow.com/>

- [28] I. Loshchilov and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," in International Conference on Learning Representations (ICLR), 2017.
- [29] N. Bodla, B. Singh, R. Chellappa, and L. S. Davis, "Soft-NMS — Improving Object Detection With One Line of Code," in Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2017, pp. 5561–5569.
- [30] World Health Organization, "Global Status Report on Violence Prevention," Geneva, Switzerland: WHO Press, 2014.